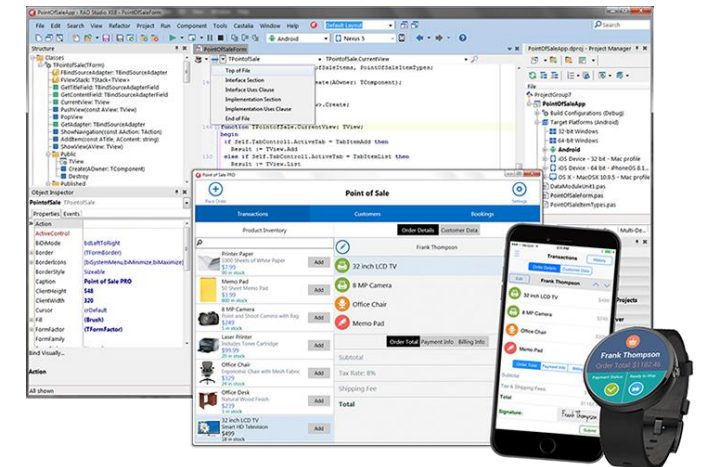
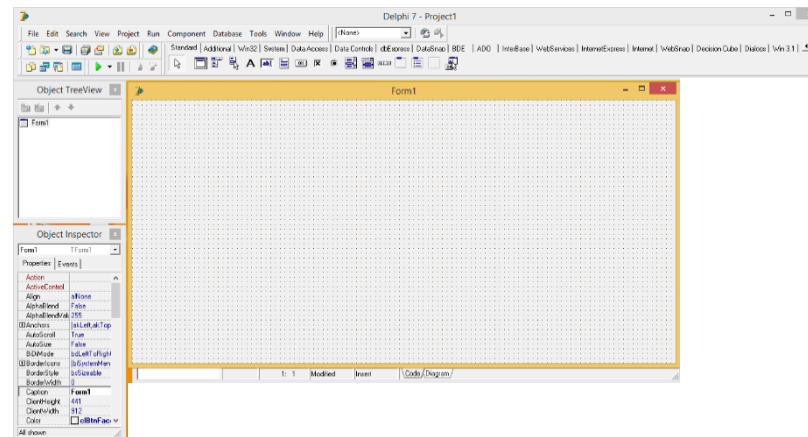
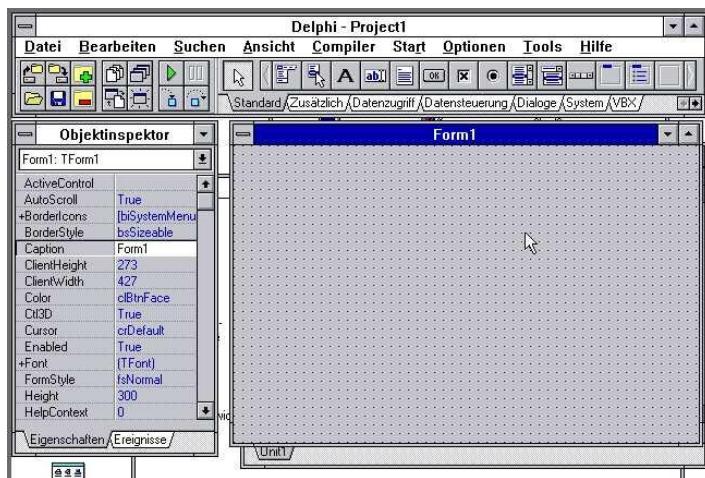


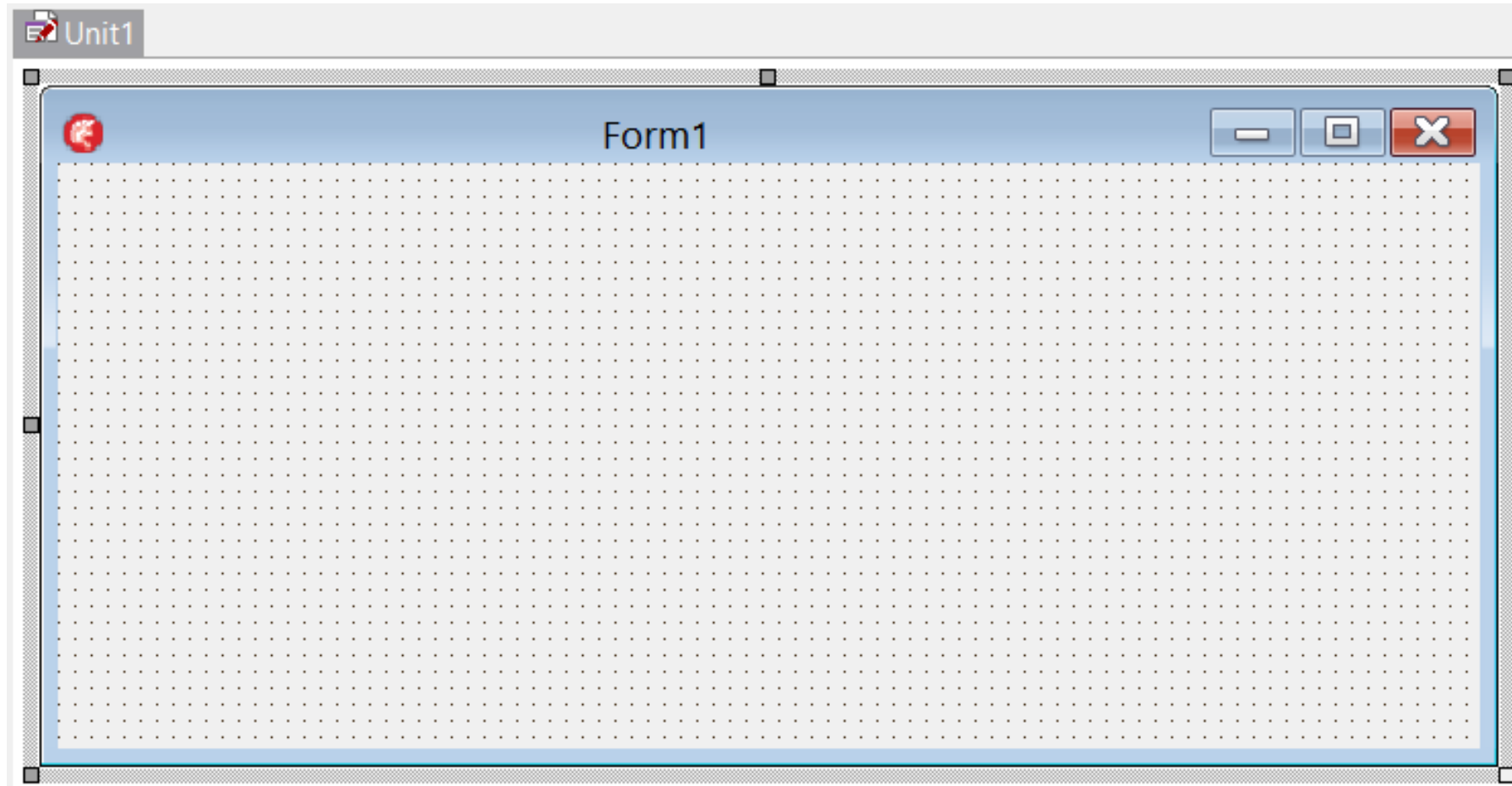
CONHECENDO UMA UNIT

Ewerton J. Silva

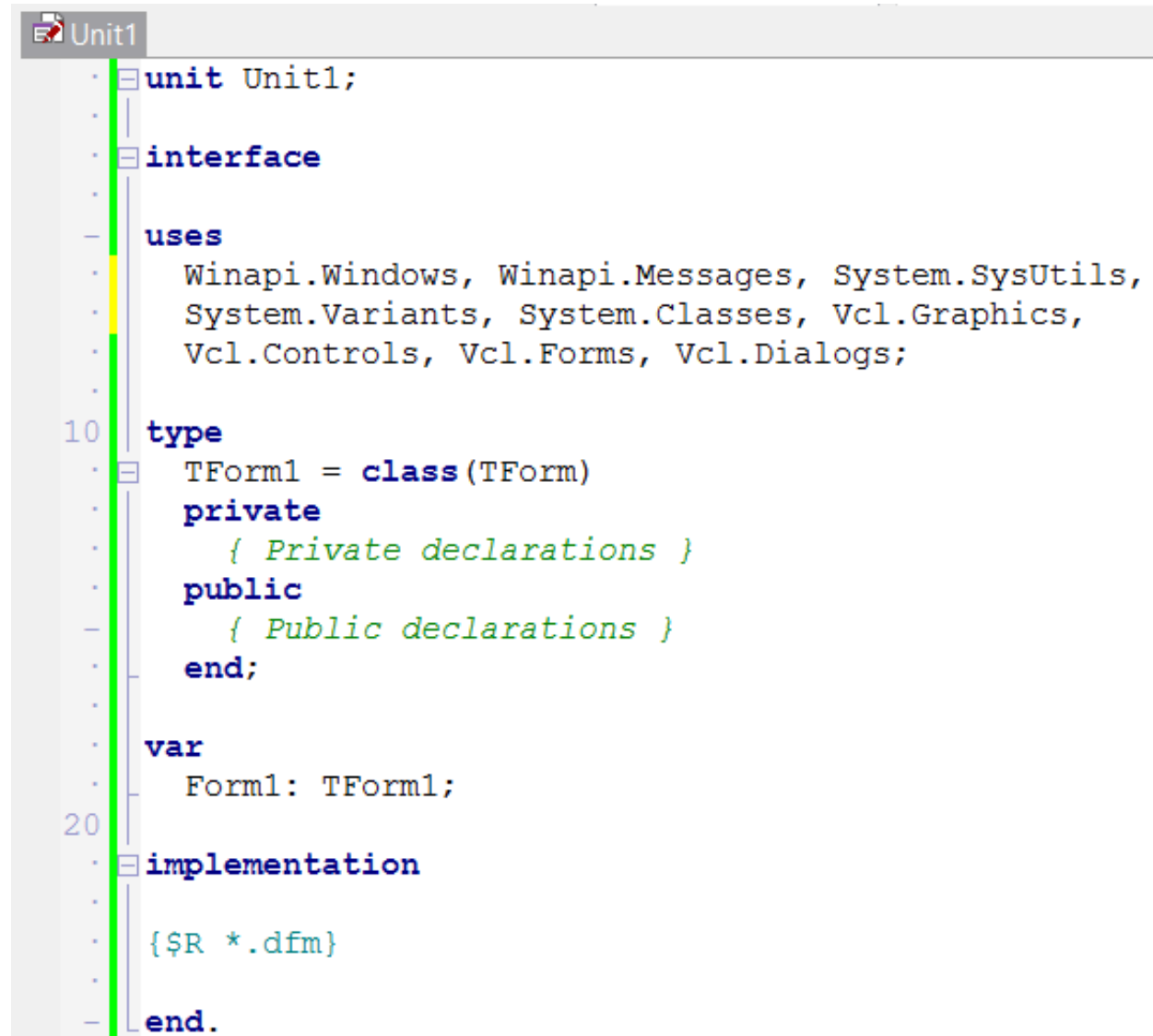
ETEC Prof. Massuyuki Kawano



Form



Editor de Códigos



```
Unit1
unit Unit1;

interface

uses
  Winapi.Windows, Winapi.Messages, System.SysUtils,
  System.Variants, System.Classes, Vcl.Graphics,
  Vcl.Controls, Vcl.Forms, Vcl.Dialogs;

10 type
  TForm1 = class(TForm)
    private
      { Private declarations }
    public
      { Public declarations }
    end;

    var
      Form1: TForm1;

20 implementation

  {$R *.dfm}

end.
```

Unit

- Define o nome do arquivo onde o código fica gravado, ela será um arquivo .pas, o formulário tem o nome .dfm

```
unit Unit1;
```

```
interface
```

```
uses
```

```
Winapi.Windows, Winapi.Messages, System.SysUtils,  
System.Variants, System.Classes, Vcl.Graphics,  
Vcl.Controls, Vcl.Forms, Vcl.Dialogs;
```

```
type
```

```
TForm1 = class(TForm)
```

```
private
```

```
{ Private declarations }
```

```
public
```

```
{ Public declarations }
```

```
end;
```

```
var
```

```
Form1: TForm1;
```

```
implementation
```

```
{ $R *.dfm }
```

```
end.
```

interface

- É a área utilizada para realizar declarações de variáveis, constantes, tipos, procedimentos, funções, etc. As declarações posicionadas nesta área poderão ser compartilhadas por outras Units ou pelo módulo principal do programa.
- Aqui só podem aparecer declarações, nunca linhas de comando.
- Faz parte da interface: USES, TYPE e VAR

```
unit Unit1;
```

```
interface
```

```
uses
```

```
Winapi.Windows, Winapi.Messages, System.SysUtils,  
System.Variants, System.Classes, Vcl.Graphics,  
Vcl.Controls, Vcl.Forms, Vcl.Dialogs;
```

```
type
```

```
TForm1 = class(TForm)
```

```
private
```

```
{ Private declarations }
```

```
public
```

```
{ Public declarations }
```

```
end;
```

```
var
```

```
Form1: TForm1;
```

```
implementation
```

```
{ $R *.dfm }
```

```
end.
```

uses

- Serve para tornar acessíveis as variáveis, constantes, procedimentos e funções declarados na área Interface de um Unit
- Conforme vamos adicionando componentes no programa, eles são adicionados automaticamente em uses.

```
unit Unit1;
```

```
interface
```

```
uses
```

```
Winapi.Windows, Winapi.Messages, System.SysUtils,  
System.Variants, System.Classes, Vcl.Graphics,  
Vcl.Controls, Vcl.Forms, Vcl.Dialogs;
```

```
type
```

```
TForm1 = class(TForm)
```

```
private
```

```
{ Private declarations }
```

```
public
```

```
{ Public declarations }
```

```
end;
```

```
var
```

```
Form1: TForm1;
```

```
implementation
```

```
{ $R *.dfm }
```

```
end.
```

type

- Trás uma classe usado no formulário e todos seus atributos
- Abaixo são listados todos os componente usados no Formulário

```
unit Unit1;

interface

uses
  Winapi.Windows, Winapi.Messages, System.SysUtils,
  System.Variants, System.Classes, Vcl.Graphics,
  Vcl.Controls, Vcl.Forms, Vcl.Dialogs;

type
  TForm1 = class(TForm)
  private
    { Private declarations }
  public
    { Public declarations }
  end;

var
  Form1: TForm1;

implementation

{$R *.dfm}

end.
```

Declarações

- Visibilidade das variáveis

```
unit Unit1;

interface

uses
  Winapi.Windows, Winapi.Messages, System.SysUtils,
  System.Variants, System.Classes, Vcl.Graphics,
  Vcl.Controls, Vcl.Forms, Vcl.Dialogs;

type
  TForm1 = class(TForm)
    private
      { Private declarations }
    public
      { Public declarations }
    end;

var
  Form1: TForm1;

implementation

{$R *.dfm}

end.
```


var

- As variáveis declaradas abaixo são globais. São iniciadas automaticamente e podem ser acessadas por qualquer Unit do programa.
- Abaixo de VAR também podemos declarar as constantes usando antes o identificados const

```
unit Unit1;

interface

uses
  Winapi.Windows, Winapi.Messages, System.SysUtils,
  System.Variants, System.Classes, Vcl.Graphics,
  Vcl.Controls, Vcl.Forms, Vcl.Dialogs;

type
  TForm1 = class(TForm)
  private
    { Private declarations }
  public
    { Public declarations }
  end;

var
  Form1: TForm1;

implementation

{$R *.dfm}

end.
```

implementation

- É a área onde fica a implementação dos procedimentos e funções.
- Basicamente é o local onde fica o código do sistema, como se fosse o início no Visualg
- Aqui também podemos declarar variáveis, constantes, tipos, etc. Porém eles só irão funcionar dentro do bloco de comando que foram declarados e não podem ser acessados em outros blocos, mesmo sendo da mesma unit e nem por outras units

```
unit Unit1;

interface

uses
  Winapi.Windows, Winapi.Messages, System.SysUtils,
  System.Variants, System.Classes, Vcl.Graphics,
  Vcl.Controls, Vcl.Forms, Vcl.Dialogs;

type
  TForm1 = class(TForm)
  private
    { Private declarations }
  public
    { Public declarations }
  end;

var
  Form1: TForm1;

implementation

{$R *.dfm}

end.
```

(\$R *.dfm)

- Faz a associação da unit com seu respectivo form e não deve ser modificado.
- Uma unit de funções não possui esta diretiva.

```
unit Unit1;

interface

uses
  Winapi.Windows, Winapi.Messages, System.SysUtils,
  System.Variants, System.Classes, Vcl.Graphics,
  Vcl.Controls, Vcl.Forms, Vcl.Dialogs;

type
  TForm1 = class(TForm)
  private
    { Private declarations }
  public
    { Public declarations }
  end;

var
  Form1: TForm1;

implementation

{$R *.dfm}

end.
```

End.

- Final do código, tudo que estiver escrito depois é ignorado.

```
unit Unit1;

interface

uses
  Winapi.Windows, Winapi.Messages, System.SysUtils,
  System.Variants, System.Classes, Vcl.Graphics,
  Vcl.Controls, Vcl.Forms, Vcl.Dialogs;

type
  TForm1 = class(TForm)
  private
    { Private declarations }
  public
    { Public declarations }
  end;

var
  Form1: TForm1;

implementation

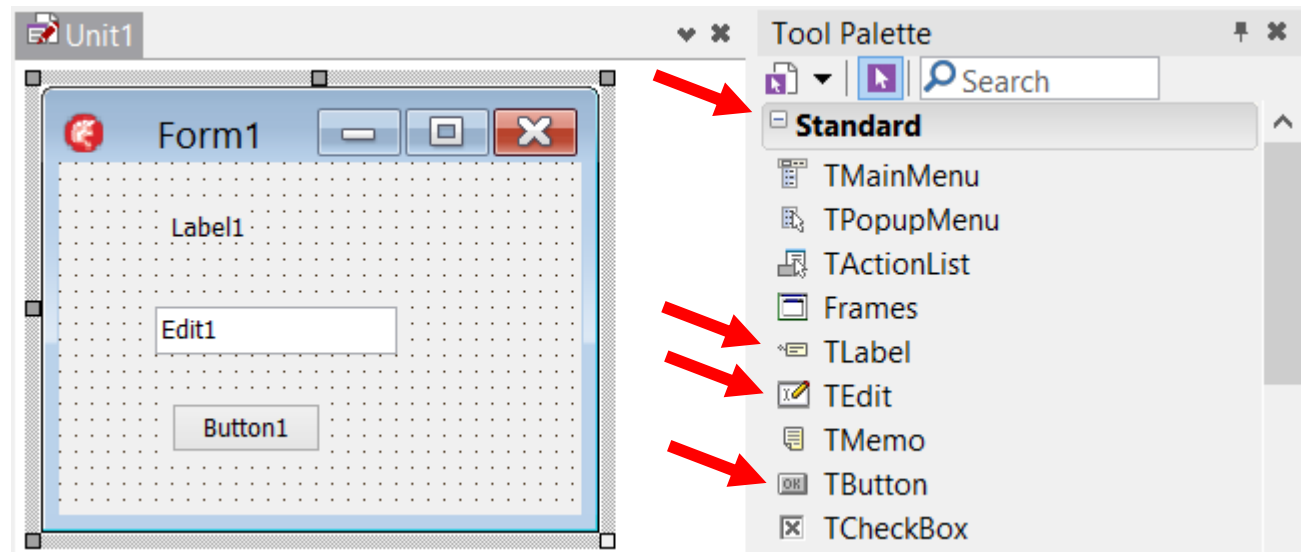
{$R *.dfm}

end.
```

Inserção de componentes no form – Ex 1

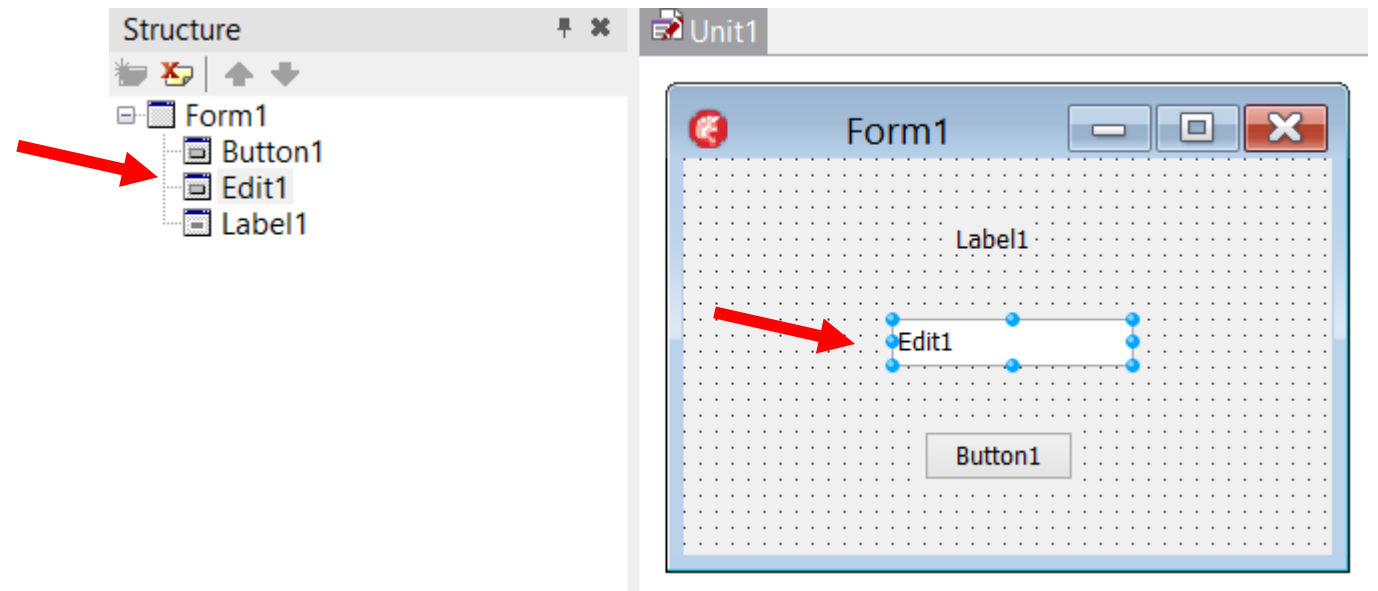
Para inserir componentes no formulário é só ir até a Tool Palette, selecionar a paleta desejada, escolher o componente que será utilizado, clicar sobre ele e em seguida clicar sobre o form.

Agora inserira três componentes no formulário: 1 Label, 1 Edit e 1 Button (Todos estão localizados na aba Standart)



Inserção de componentes no form

- Note que em Structure aparecem todos os componentes inseridos, neste exemplo o Edit1 está selecionado no Form e no Structure.
- Desta maneira é possível identificar ou selecionar componentes em qualquer um destes locais.



Código após a inserção de componentes no Form

- Pressione F12 para verificar as alterações que ocorreram na Unit
- No exemplo foram adicionados no form um button, um label e um edit, note que eles foram agrupados logo abaixo da classe referente ao form usado.

```
unit Unit1;

interface

uses
  Winapi.Windows, Winapi.Messages, System.SysUtils,
  System.Variants, System.Classes, Vcl.Graphics,
  Vcl.Controls, Vcl.Forms, Vcl.Dialogs, Vcl.StdCtrls;

type
  TForm1 = class(TForm)
    Label1: TLabel;
    Edit1: TEdit;
    Button1: TButton;
  private
    { Private declarations }
  public
    { Public declarations }
  end;

var
  Form1: TForm1;

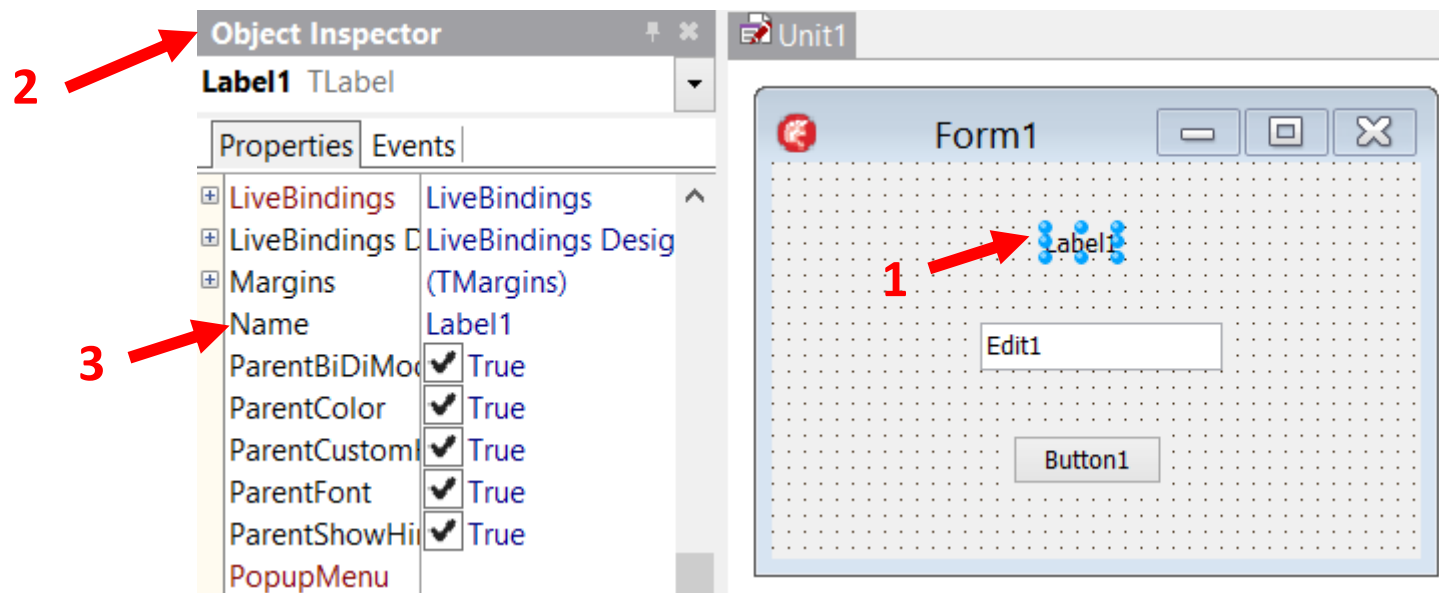
implementation

{$R *.dfm}

end.
```

Renomeando os componentes

- É muito importante lembrar sempre de renomear o componente assim que ele é adicionado. Isto ajuda o programador a identificar a real função daquele objeto no programa.
- Para renomear um componente 1- selecione-o e em seguida procure no 2- Object Inspector pela propriedade 3- Name .



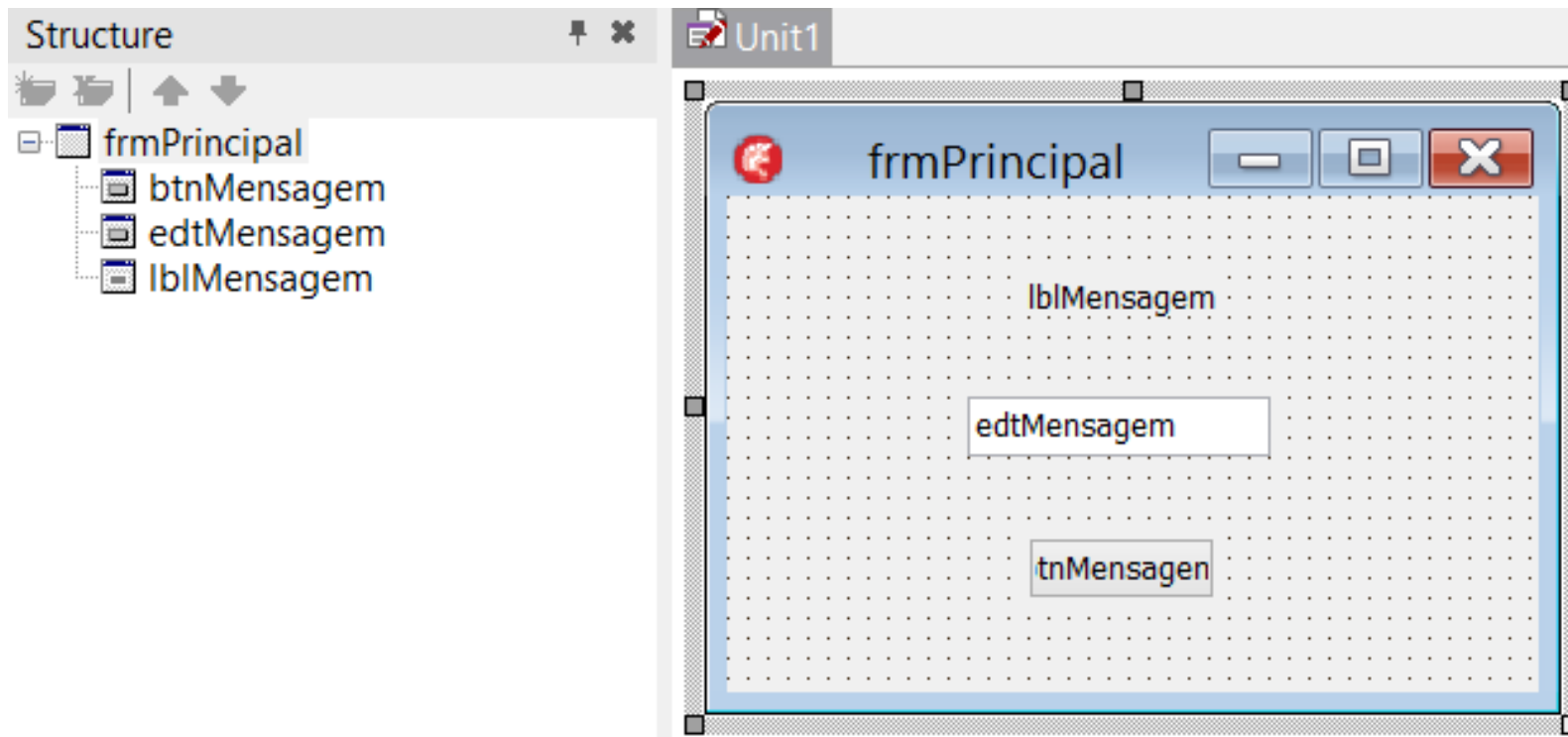
Renomeando os componentes

- Os nomes dos componentes não podem conter acentos, espaços ou caracteres especiais.
- Renomeie todos os componentes conforme a tabela abaixo:

Nome Atual	Novo Nome
Form1	frmPrincipal
Button1	btnExibeMsg
Edit1	edtMensagem
Label1	lblMensagem

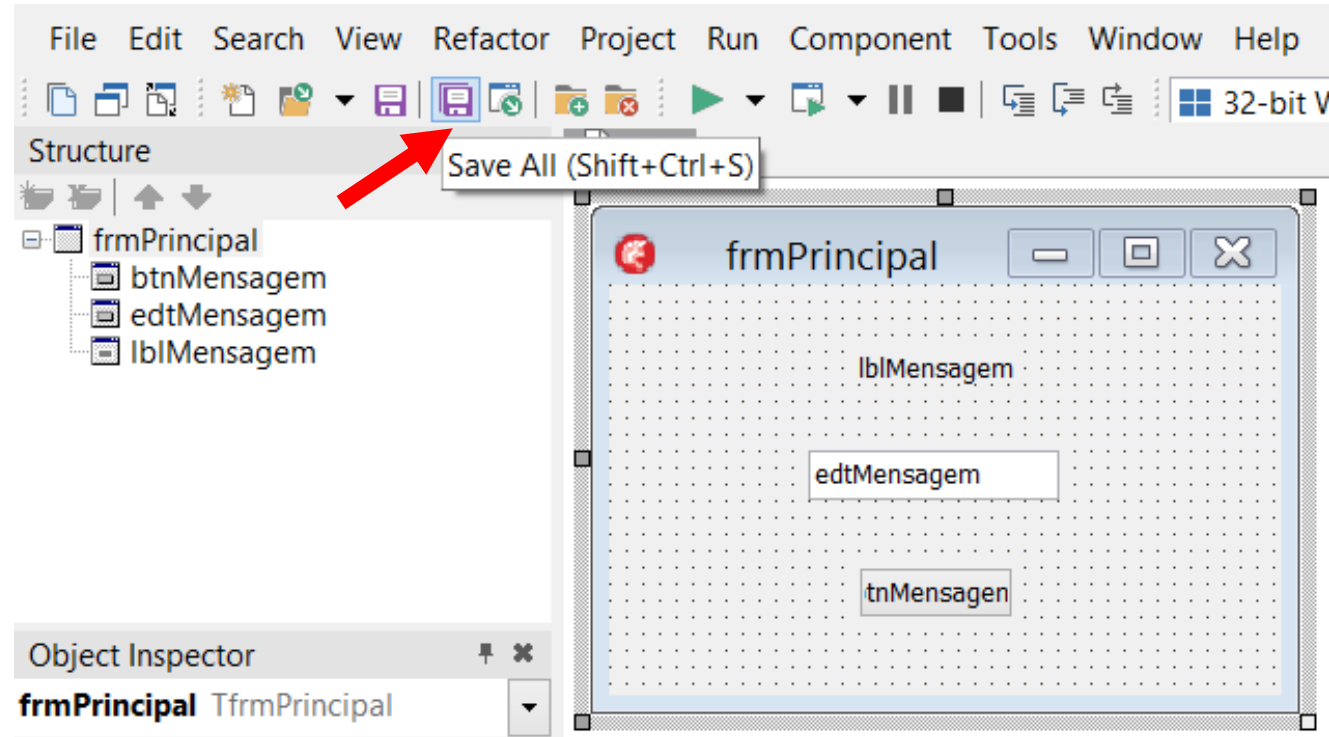
Renomeando os componentes

Veja que imediatamente a os nomes são atualizados.



Salvando o primeiro programa

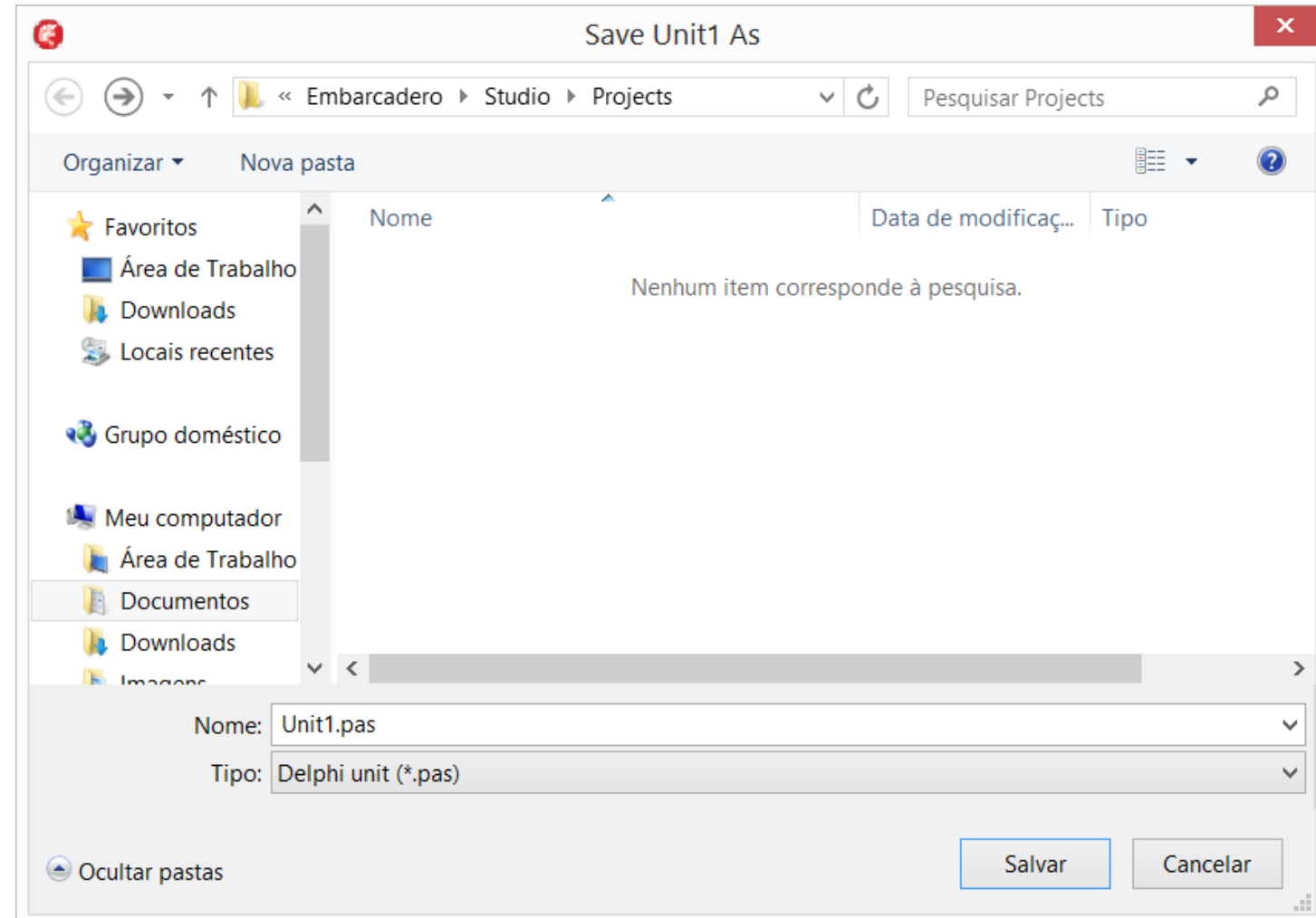
Para salvar o programa pela primeira vez é recomendado que seja utilizada a opção Save All, pois desta maneira iremos salvar o programa completo.



Salvando o primeiro programa

Veja que o Delphi sugere automaticamente uma pasta para que o programa seja salvo, o ideal é que você lembre-se de selecionar sempre o local onde o arquivo será salvo.

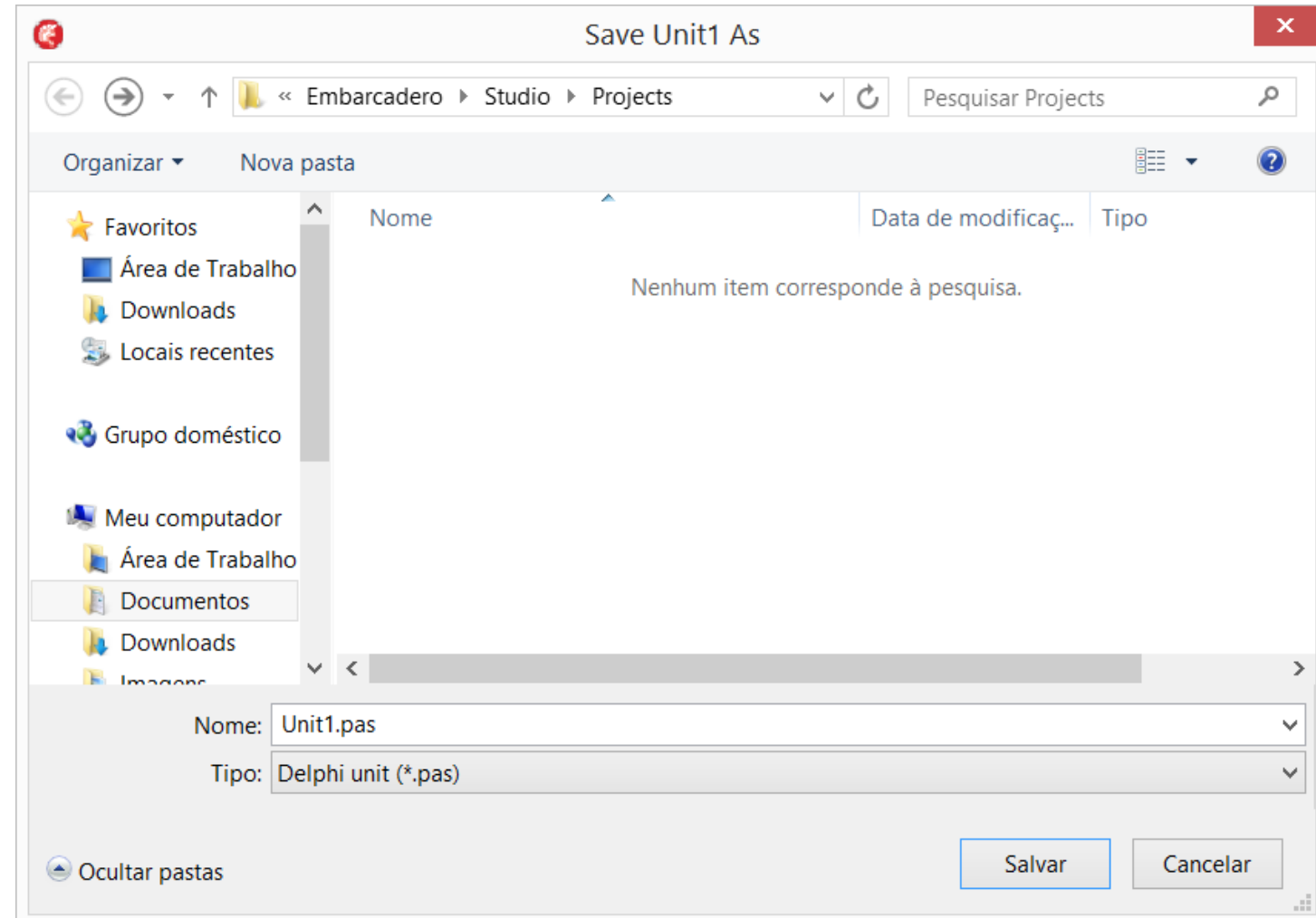
Se o projeto for salvo em um local e a Unit em outro você terá que lembrar onde estão os dois arquivos para conseguir fazer o programa rodar.



Salvando o primeiro programa

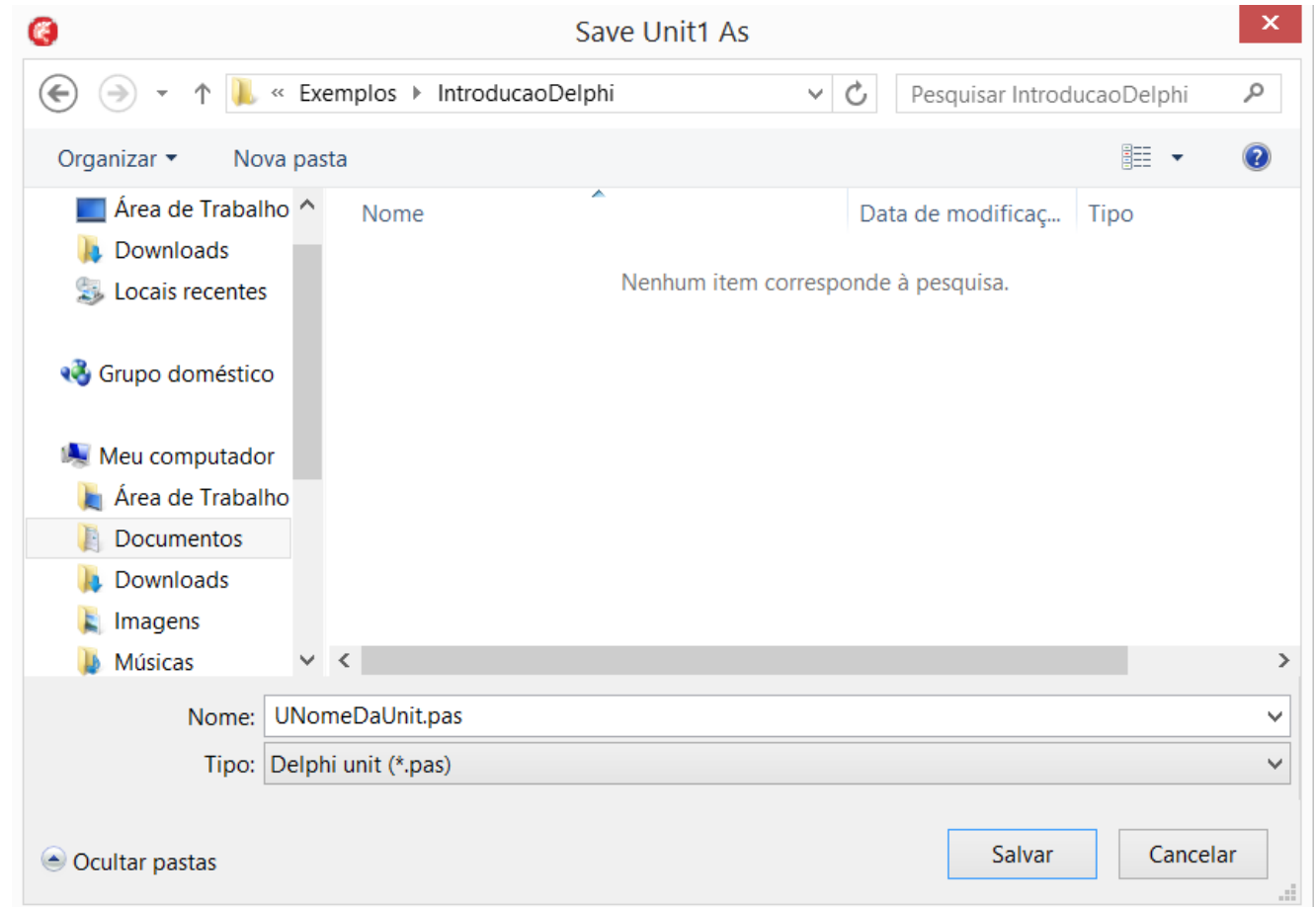
Primeiramente iremos salvar a Unit do programa. Como sugestão eu recomendo que para facilitar a identificação sempre comece o nome de uma Unit com a letra “U”.

Lembre-se que é possível verificar o tipo do arquivo logo abaixo do nome.



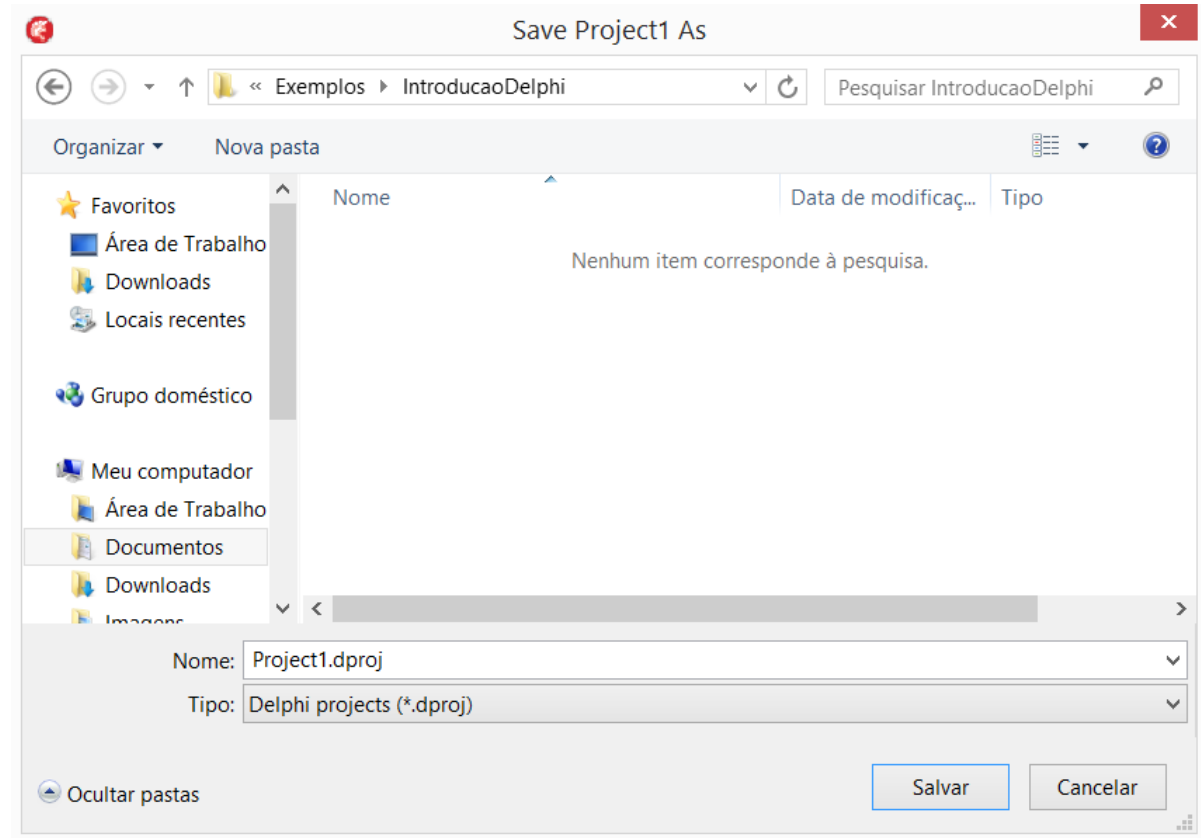
Salvando o primeiro programa

- Para facilitar o entendimento vamos salvar desta vez a Unit com o nome UNomeDaUnit



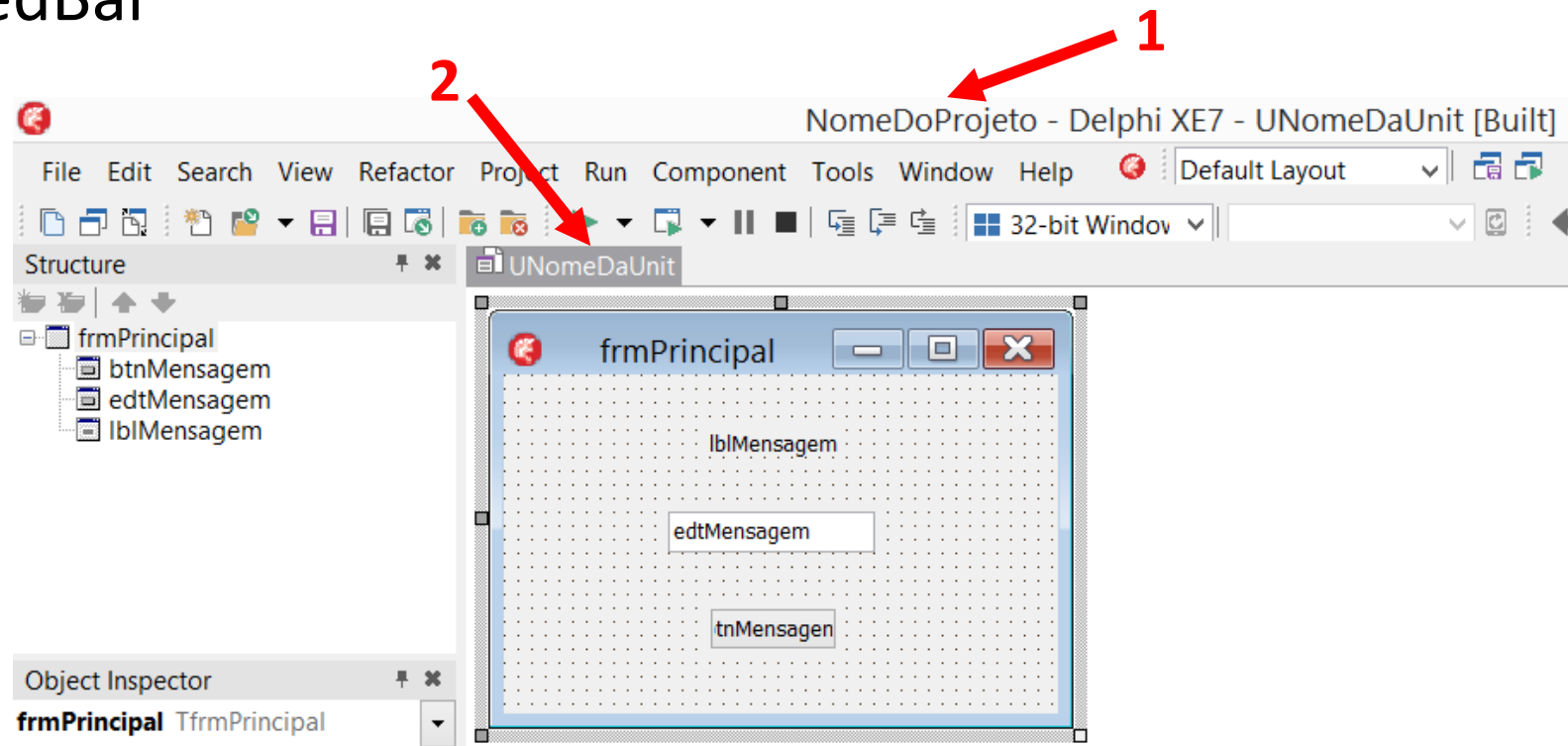
Salvando o primeiro programa

- Como clicamos em Save All, iremos salvar todo o programa, o que incluem a Unit e o arquivo do projeto, este arquivo é necessário para que o programa possa ser executado, por isso é muito importante que ele seja salvo no mesmo local que está a Unit
- Vamos salvá-lo com o nome “NomeDoProjeto”



Salvando o primeiro programa

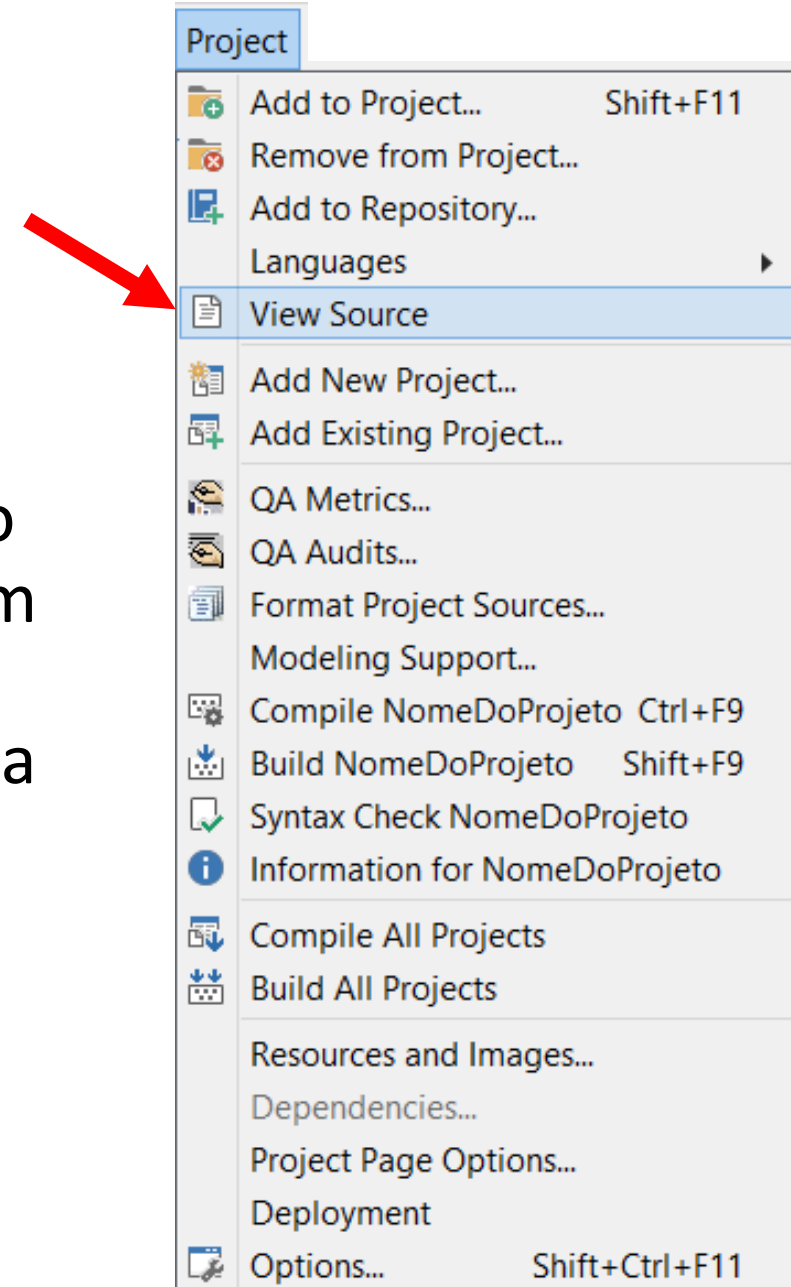
- Agora que nosso programa está salvo é possível visualizar o 1- nome do projeto na barra de títulos e o nome da 2- Unit em uma aba abaixo da SpeedBar



Trabalhando com projetos

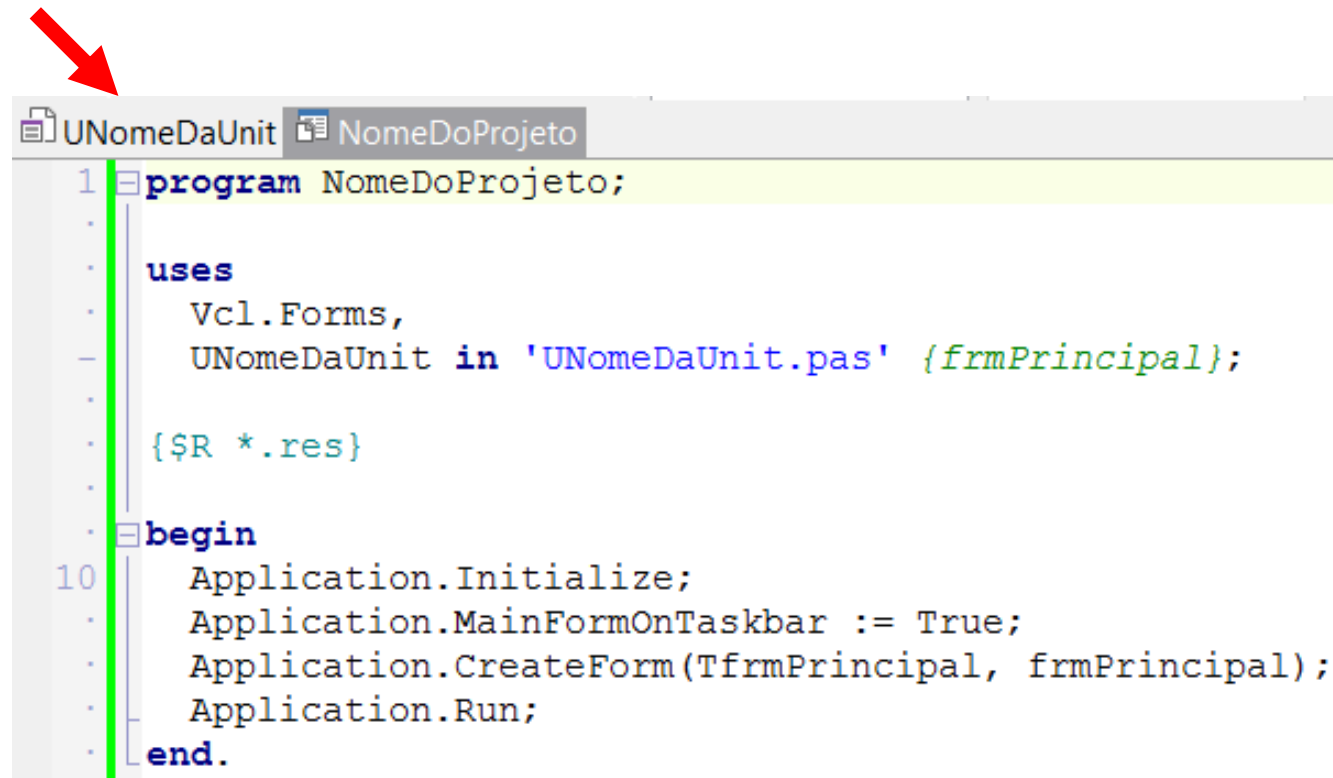
Quando iniciamos a criação de um novo aplicativo em Delphi ele atribui ao “programa principal” a extensão DPR. Esse código define que a partir desse documento o programador construirá sua aplicação. Através de um projeto podemos gerar código executáveis (EXE), no entanto grande parte da programação fica armazenada na UNIT.

O projeto pode ser visualizado através da opção do menu *Project* | *View Source*.



Código do projeto

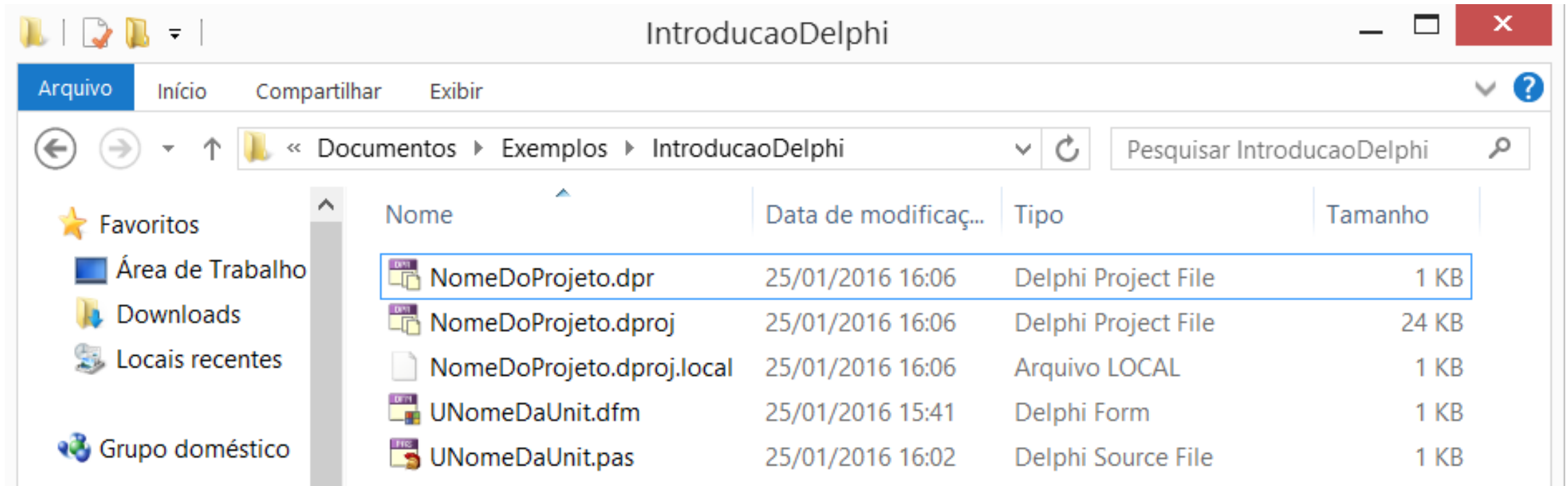
- Para voltar na Unit do formulário basta clicar sobre a mesma.



```
1 program NomeDoProjeto;
.
.
. uses
.   Vcl.Forms,
-   UNomeDaUnit in 'UNomeDaUnit.pas' {frmPrincipal};
.
. {$R *.res}
.
. begin
10 Application.Initialize;
.   Application.MainFormOnTaskbar := True;
.   Application.CreateForm(TfrmPrincipal, frmPrincipal);
.   Application.Run;
. end.
```

Arquivos do Delphi

Ao iniciarmos um projeto, o Delphi apresenta forms e units, porém em um projeto temos vários arquivos diferentes.



Arquivos do Delphi

EXTENSÃO	DESCRIÇÃO
DPR	Arquivo contendo o código fonte do projeto.
DPROJ	Arquivo contendo as configurações do projeto, como versionamento, dependências e caminhos.
RES	Arquivo binário contendo outras informações do projeto, como ícones e imagens.
IDENTCACHE	Arquivo temporário de cache.

Outros arquivos

EXTENSÃO	DESCRIÇÃO
PAS	Código fonte
DFM	Arquivo do Form, que recebe automaticamente o mesmo nome dado a sua Unit.
BPL	Borland Package Library - DLL para uso em outros projetos Delphi.
DCU	A Unit e o Form compilados, dcu significa Delphi Compiled Unit. A união de todos os .dcu de um projeto juntamente com outras Units internas do Delphi irão formar o arquivo executável .exe.
DPK	Código fonte de um pacote Delphi, normalmente utilizado em instalação de componentes do delphi.

Outros arquivos

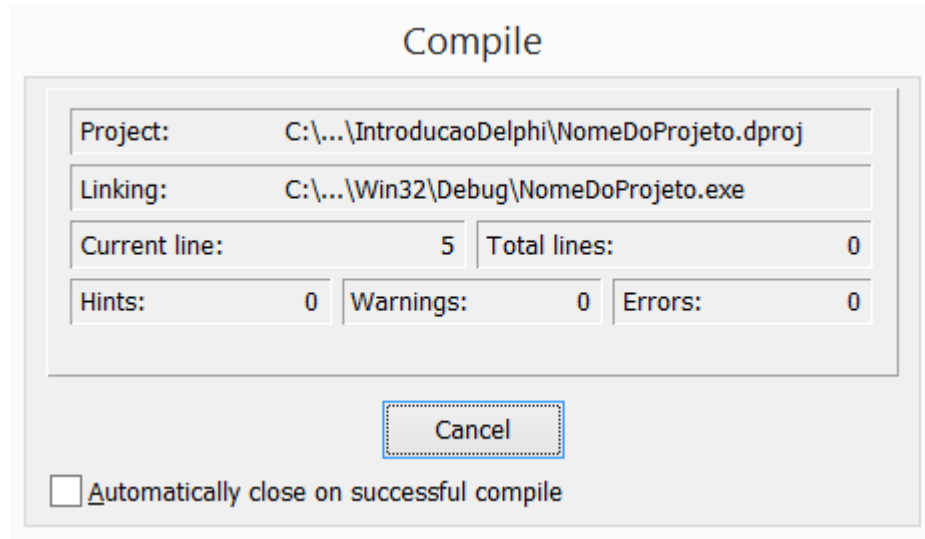
EXTENSÃO	DESCRIÇÃO
DOF e CFG	Arquivos de configuração de compilação e ambiente
~PA, ~DF, ~DP	Arquivos temporários.
DSK	Configuração de desktop.
EXE	O aplicativo independente (executável que roda sem o Delphi), é criado com o mesmo nome dado ao projeto.

Amplie seus conhecimentos em: <http://nazarenoneto.com.br/2012/01/30/extensoes-dos-arquivos-no-delphi/> e <http://www.devmedia.com.br/entendendo-as-extensoes/23271>

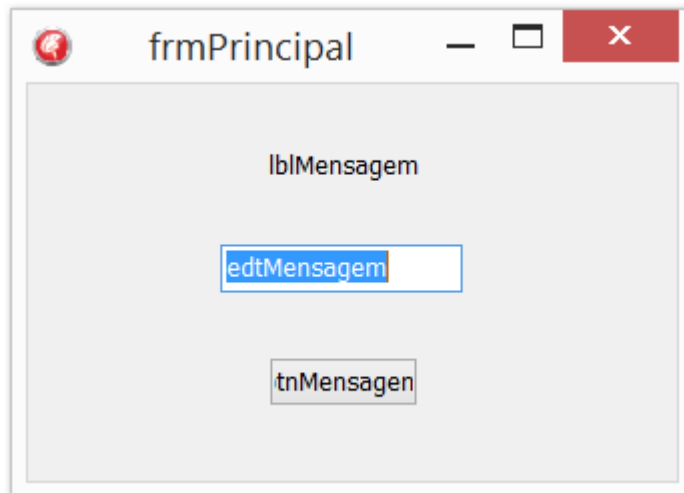
Abrindo projetos

- Para abrir seu projeto novamente, sempre abra o arquivo de projeto (**.dpr** ou **.dproj**) e o Delphi irá automaticamente carregar todos as units e forms adicionados ao seu projeto. É possível visualizar todos os arquivos do projeto na seção Project Manager.

Executando o programa

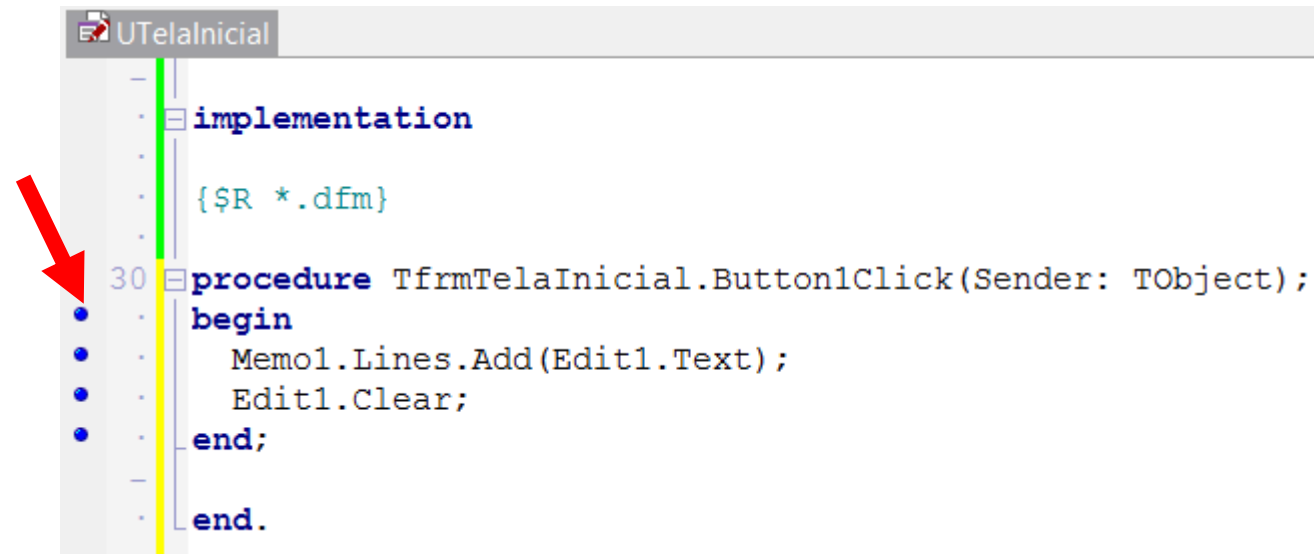
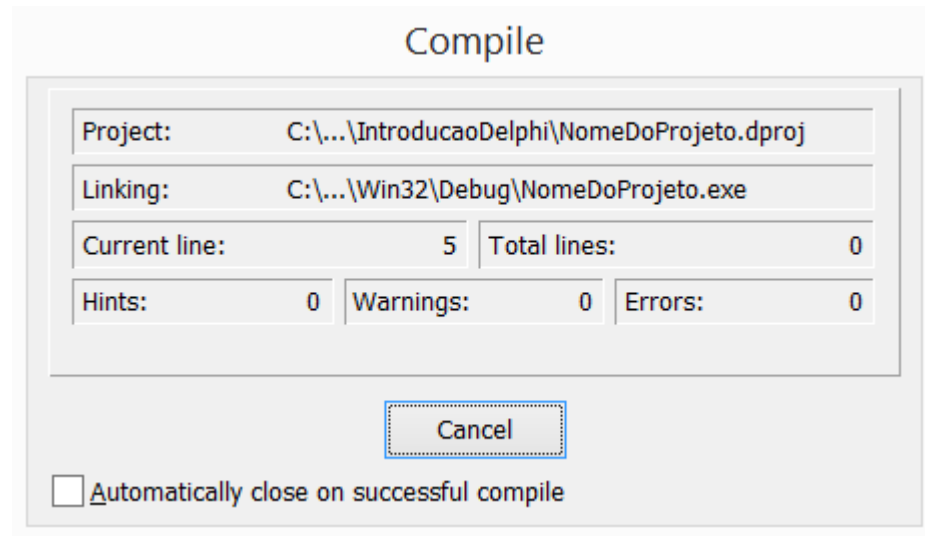


Sempre que executamos o programa pelo Delphi utilizando o atalho F9, o programa primeiro é compilado, o que gera um arquivo executável do Windows, em seguida o programa é executado.

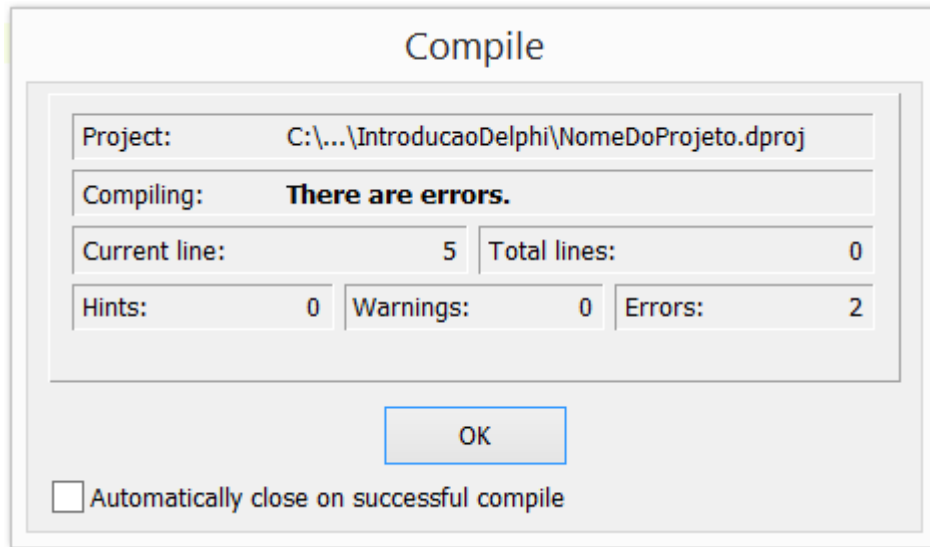


Executando o programa

É possível compilar um programa sem executá-lo, isto é muito útil para identificar erros na programação, para isto utilize a combinação de teclas CTRL + F9



Executando o programa

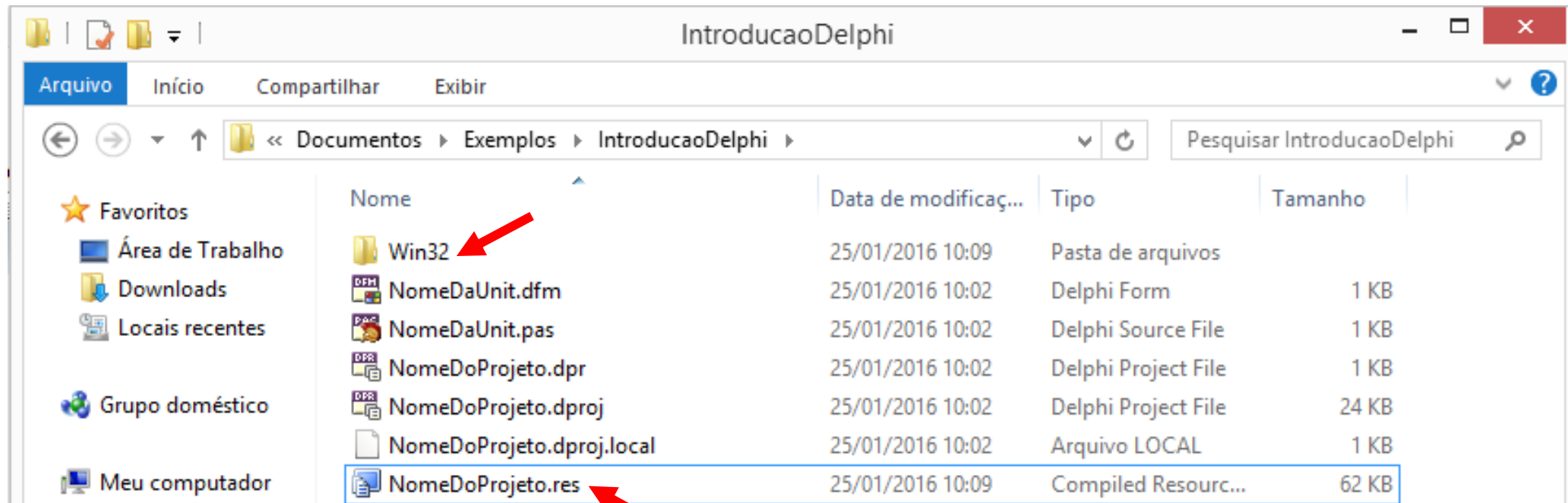


- Caso ocorra algum erro uma mensagem é exibida.
- E a linha que não pode ser executada é destacada.

```
30 procedure TfrmTelaInicial.Button1Click(Sender: TObject);  
  begin  
    Memo1.Lines.Add(Edit1.Text)  
33    Edit1.Clear;  
  end;
```

Arquivos de um projeto

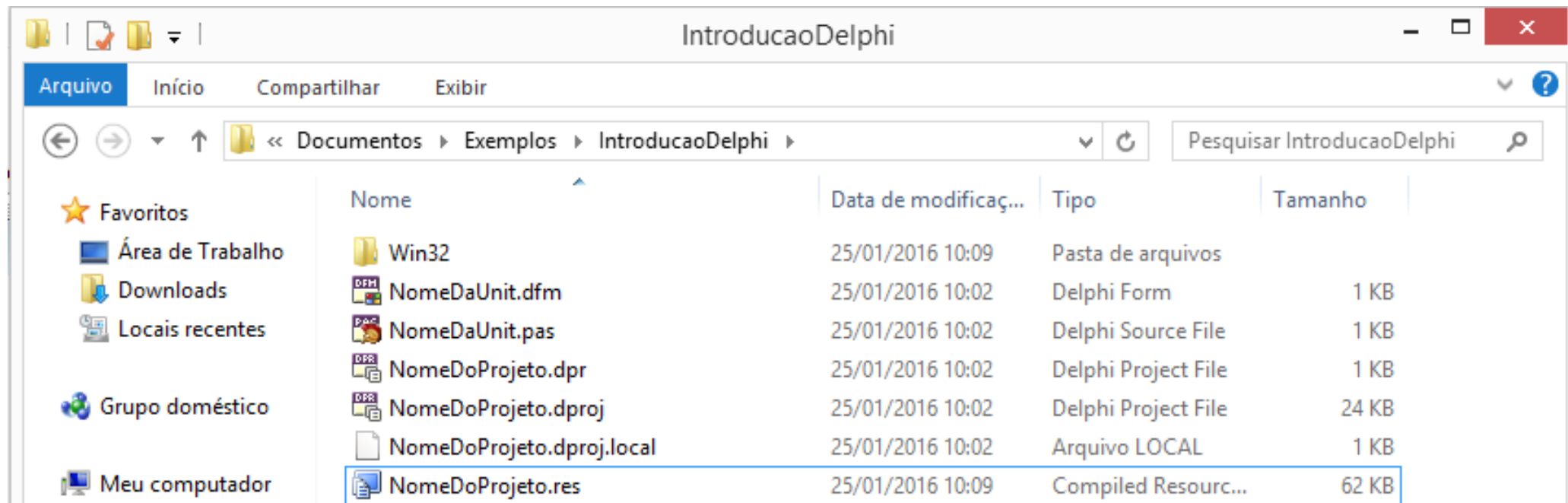
Ao compilar ou rodar um programa feito em Delphi, mais dois arquivos são criados e adicionados ao projeto.



Arquivos de um projeto

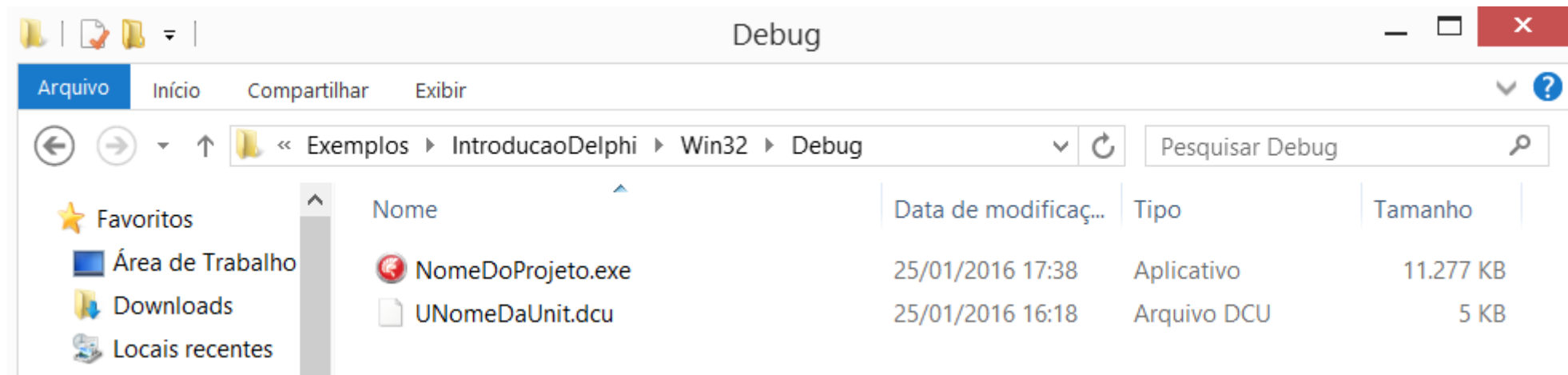
Os arquivos principais são os **.dpr**, **.pas** e **.dfm**, os demais caso não sejam encontrados serão recriados com as opções padrão.

Caso se deseje fazer uma cópia de segurança de um projeto, os outros arquivos podem ser apagados, pois podem ser recriados através de uma nova compilação do projeto. Após a compilação, apenas o executável é necessário. É ele que será enviado ao usuário final.



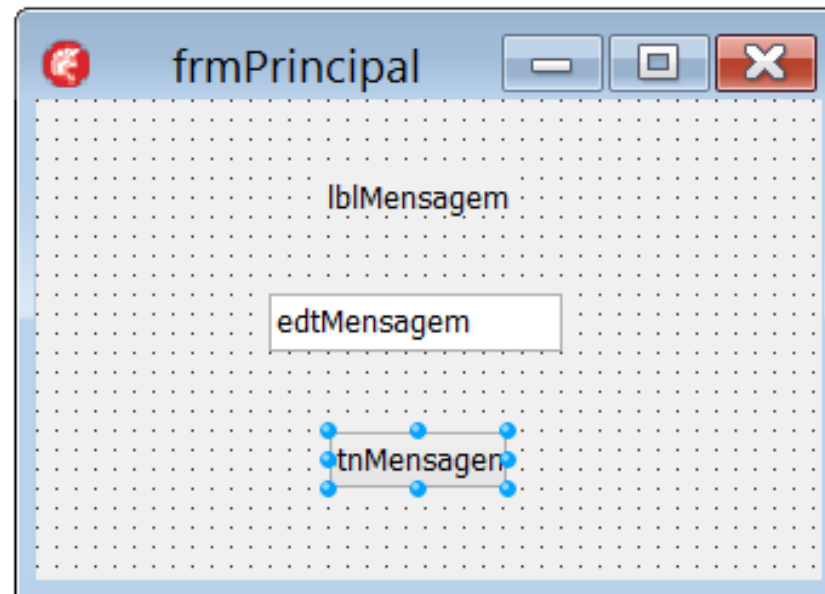
Arquivos de um projeto

Dentro da pasta Win32 existe uma pasta com o nome Debug e dentro dela está o .exe, o executável do programa, é ele que deve ser enviado ao cliente que vai executar o programa.



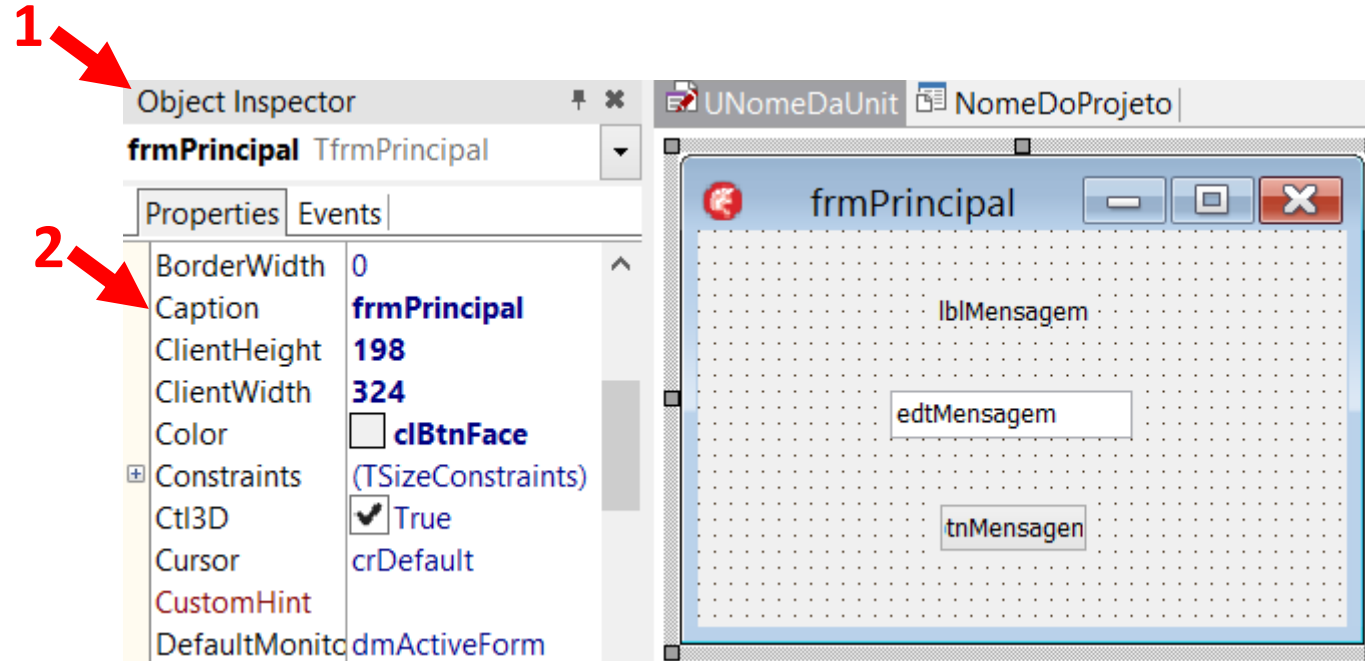
Continuando com o primeiro programa

Agora vamos melhorar o visual do programa, pois o que está sendo exibido são os nomes de cada um dos componentes, agora voltaremos as propriedades de cada um dos componentes que inserimos.



Alterando a exibição dos componentes

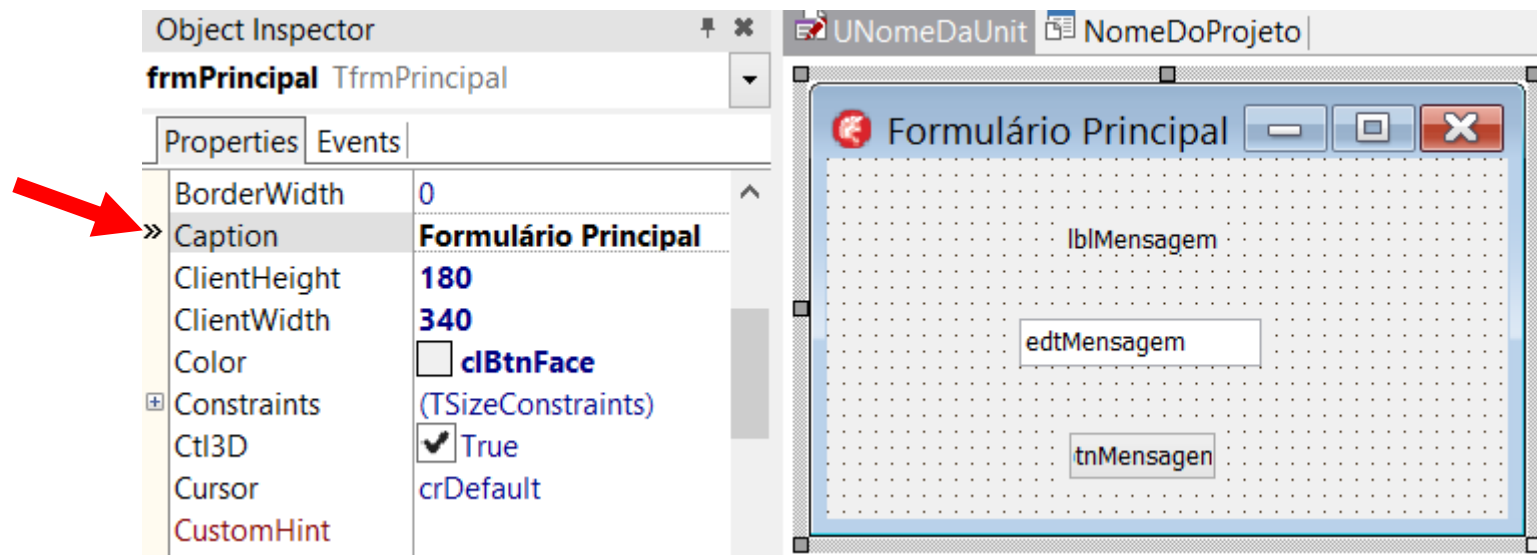
Para alterar o texto exibido no Formulário, selecione-o, e procure no 1- Object Inspector a propriedade 2- Caption e altere o nome de acordo com sua necessidade, escreva “Formulário Principal”



Alterando a exibição dos componentes

Este nome pode conter acentos, espaços e caracteres especiais, diferente do nome do componente.

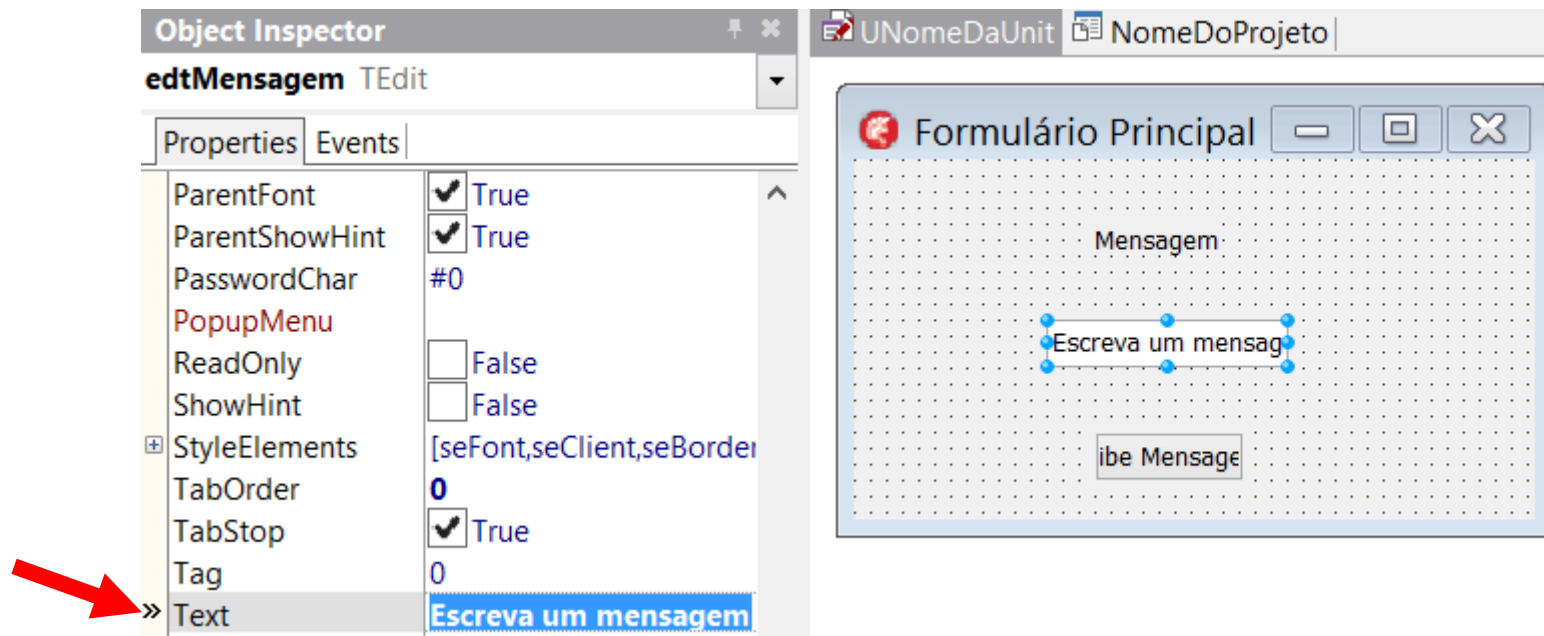
Agora altere o Caption do Label para “Mensagem” e o Caption do Button para “Exibir Mensagem”



Alterando a exibição dos componentes

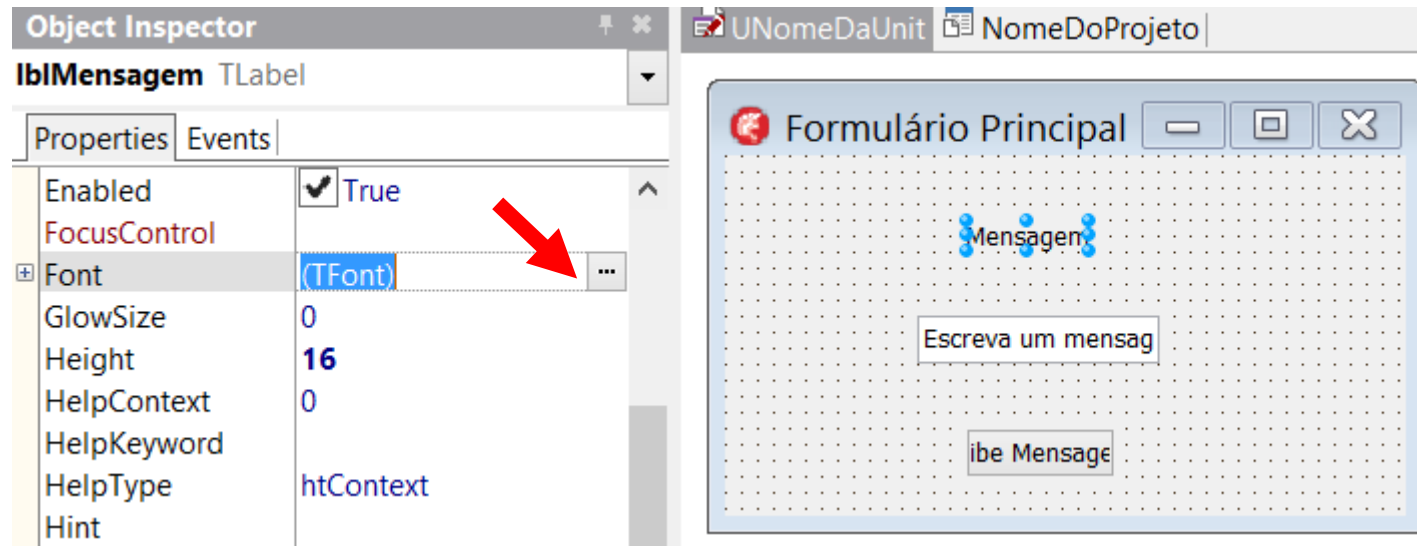
O componente Edit não tem a propriedade Caption, para alterar seu valor devemos alterar a propriedade Text.

Selecione o componente edtMensagem e altere a propriedade Text para “Escreva sua Mensagem”



Alterando a exibição dos componentes

Também é possível de um modo bem simplificado alterar as propriedades da fonte dos componente, alterando assim o modo que eles são exibidos. Selecione o componente `IblMensagem`, e clique em “...” na propriedade Font

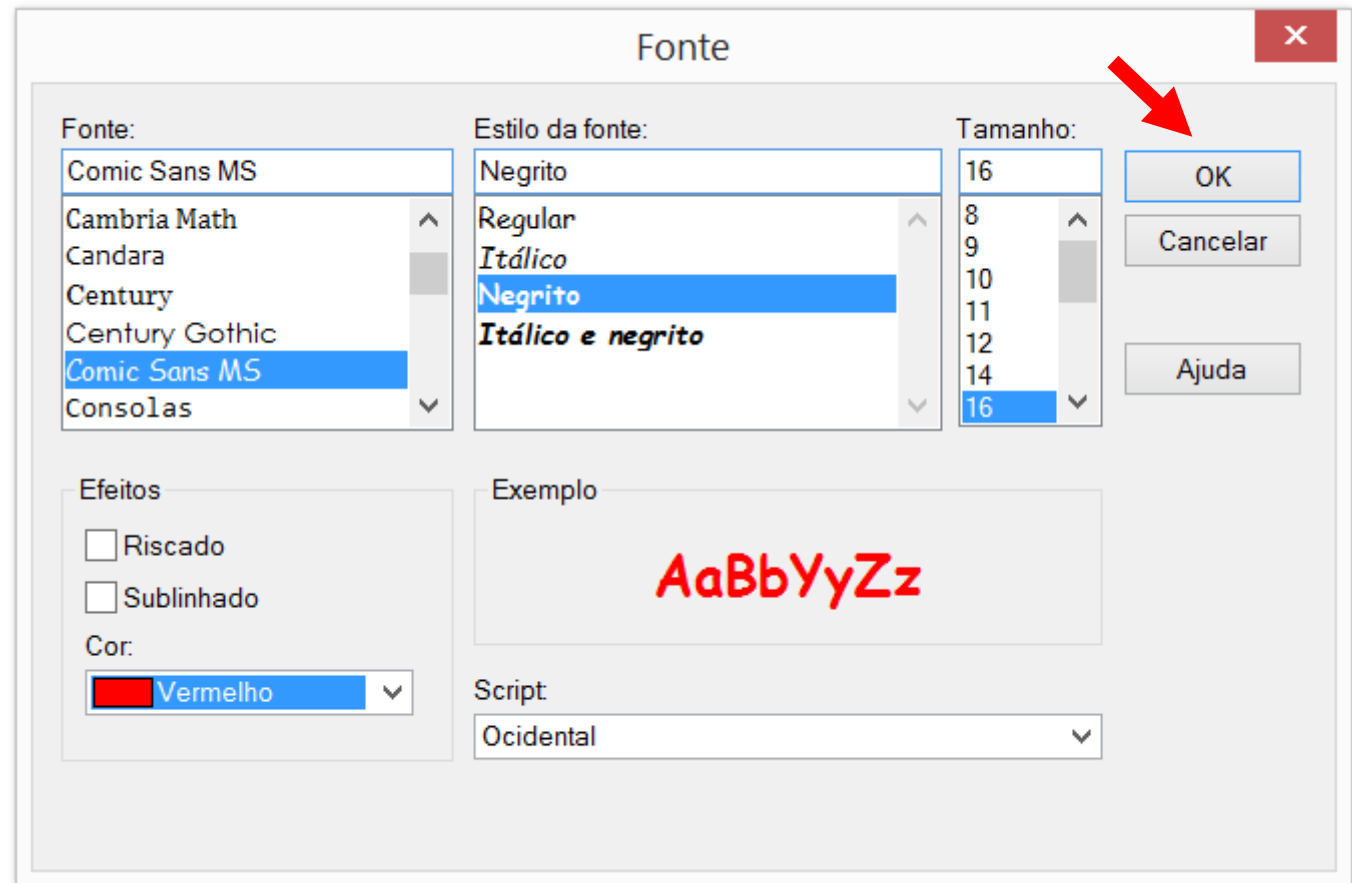


Alterando a exibição dos componentes

Aqui alteramos as propriedades da fonte como se fosse um editor de texto.

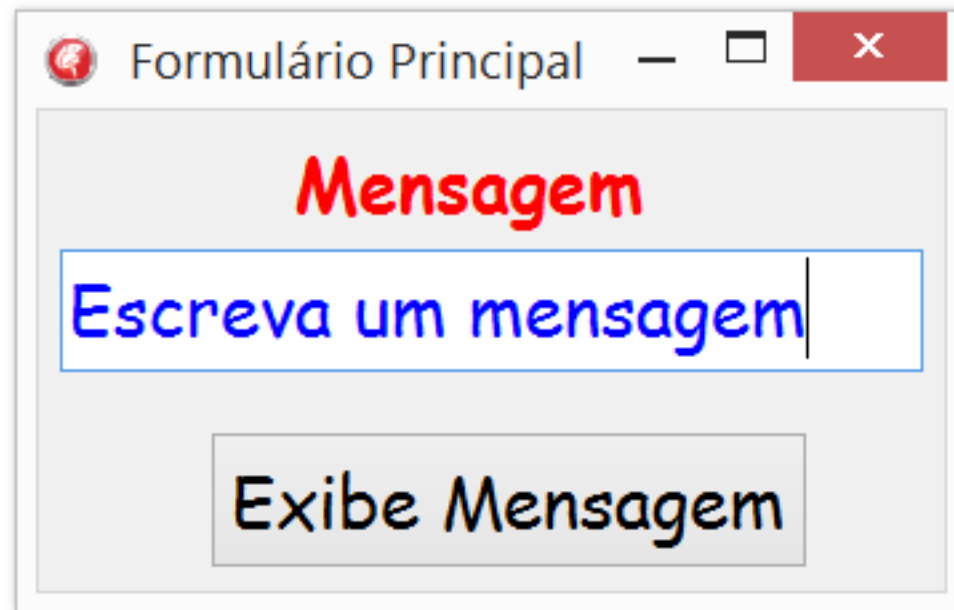
Neste exemplo selecionei a fonte “Comics Sans MS”, deixei-a em “Negrito” com o tamanho “16”, na cor “Vermelho”

Clique em Ok para aplicar as alterações.



Alterando a exibição dos componentes

Deixe sua janela como a do exemplo abaixo e execute o programa!

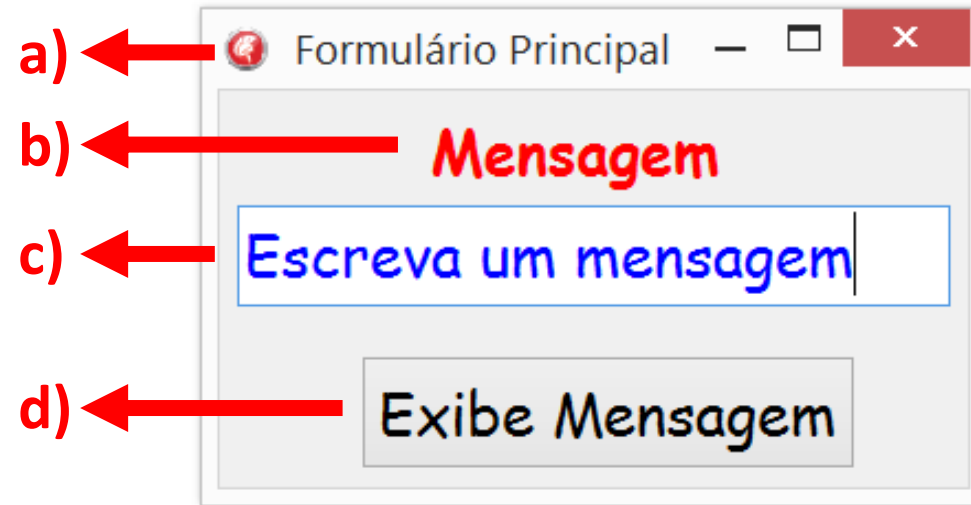


TECLAS IMPORTANTES

Tecla	Função
F12	Alterna entre o code editor e o form designer.
F11	Alterna entre o code editor, form designer e a object inspector.
F10	Torna o foco para a janela principal.
F9 (RUN)	Permite compilar e executar o projeto para testes. Este processo gera automaticamente o arquivo .EXE no diretório onde foi gravado o arquivo de projeto (.DPR).
CTRL + F9	Permite compilar o projeto sem executar. Ideal para conferência de código.
SHIFT + F12	Permite alternar entre os formulários do projeto. Equivalente ao ícone View Form na SpeedBar.
CTRL + F2	Permite 'destravar' o Delphi em caso de testes onde ocorram exceções, como veremos mais adiante.

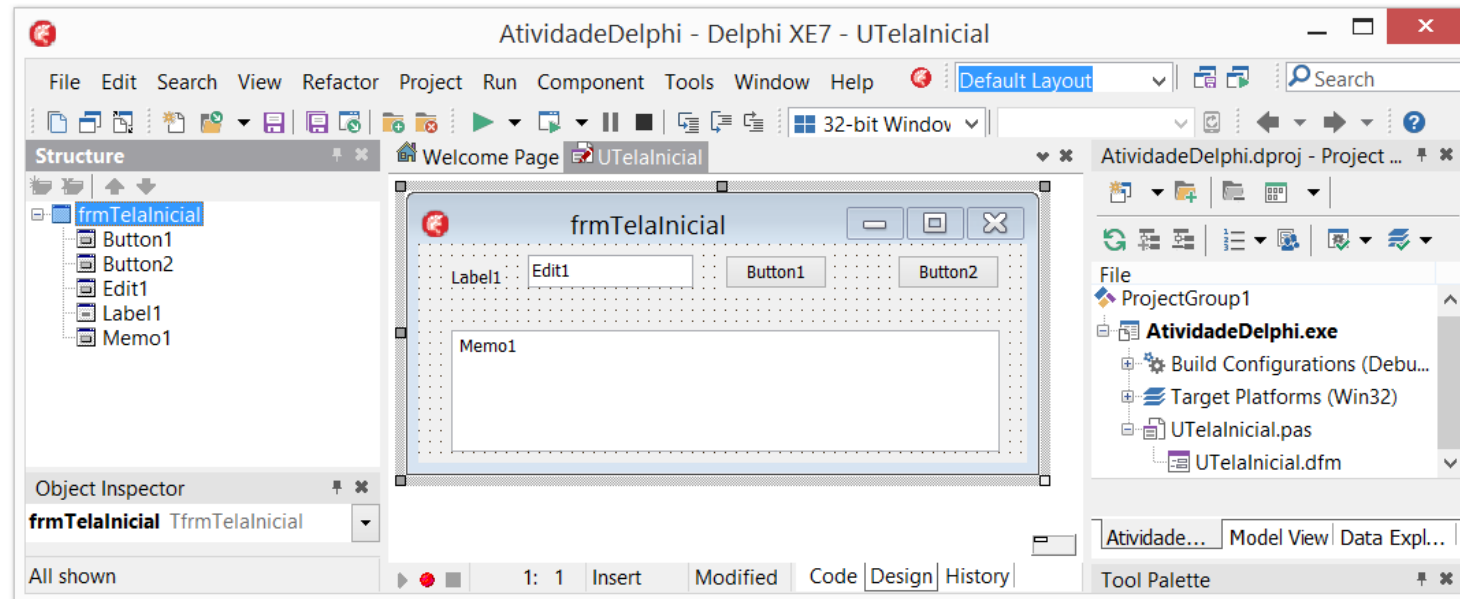
Exercícios

7. Diferencie Form e Unit.
8. Diferencie variáveis públicas, privadas e globais.
9. O que acontece com o código que fica depois do “end.”
10. Identifique na imagem abaixo quais são os componentes que aparecem e qual sua função.



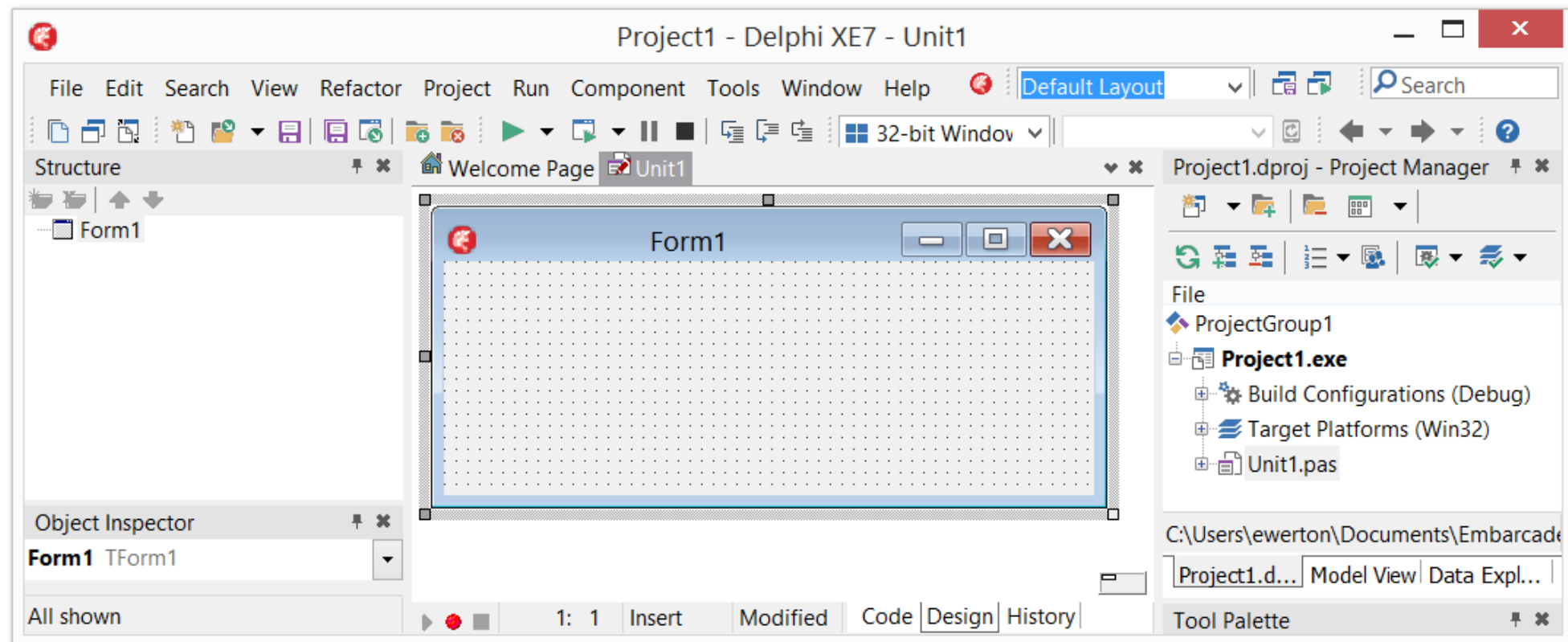
Exercícios

11. Quais são as funções das propriedades: Caption, Text, Name e Font?
12. É possível executar um programa abrindo com o Delphi apenas o Form? Porque?
13. Identifique na imagem abaixo qual componente está selecionado.



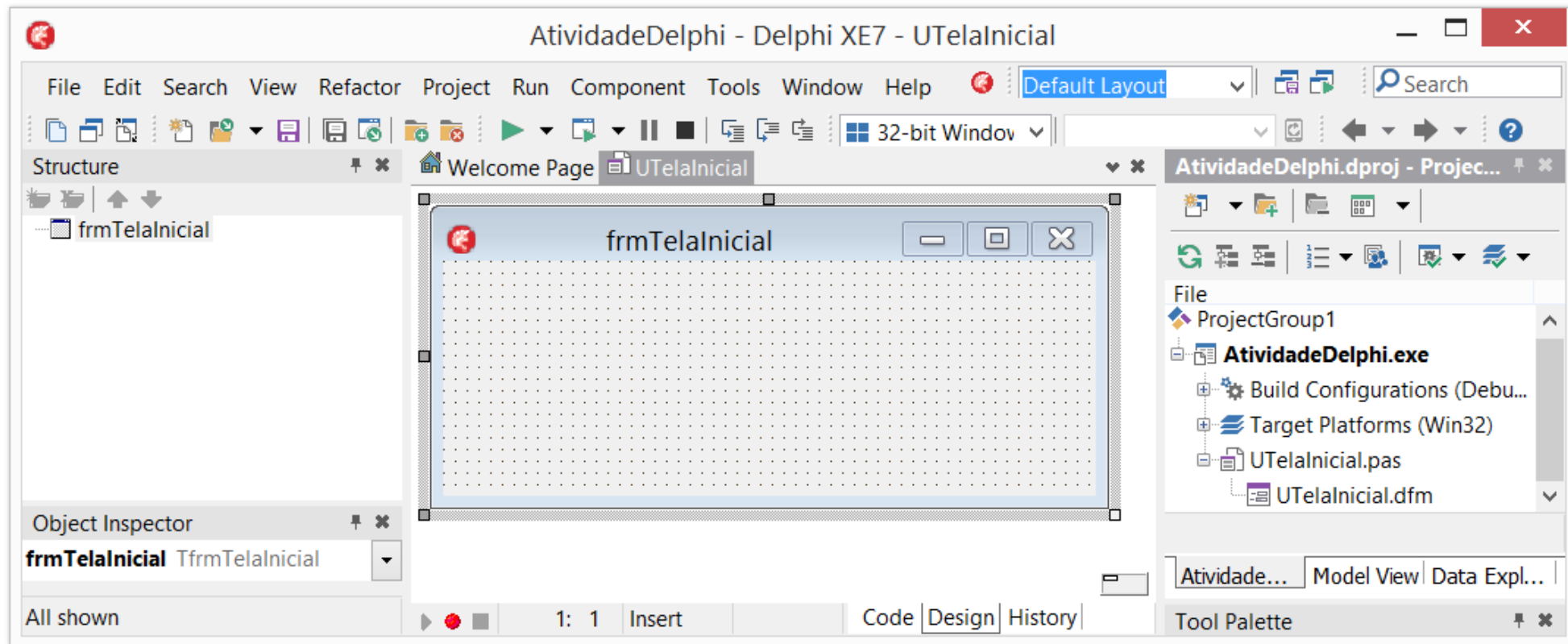
Exercícios

14. De acordo com a imagem a seguir cite qual o nome do Projeto, qual o nome da Unit e qual é o nome do Form.



Exercícios

15. De acordo com a imagem a seguir cite qual o nome do Projeto, qual o nome da Unit e qual é o nome do Form.



Exercícios

16. É necessário o Delphi para executar o arquivo .exe gerado na compilação? Porquê?
17. É possível realizar alterações com o Delphi utilizando apenas o executável?
18. Nos componentes Label, Button e Form. Em qual propriedade não é possível utilizar caracteres especiais?

Bibliografia

- ANSELMO, FERNANDO ANTONIO F. Desvendando o Caminho das Pedras. Web Publicação de 1995-97. Disponível em <http://www.greantoniobraga.seed.pr.gov.br/redeescola/escolas/13/870/10/arquivos/File/Adenildo/Biblia-Delphi-7-PtBr.pdf>. Acesso em 27/04/2015
- BARCELOS, ÉRICA. Linguagem de Programação Delphi Orientada a objeto – Web Publicação de Fevereiro de 2012. Disponível em <http://www.simonsen.br/its/pdf/apostilas/base-tecnica/2/linguagem-de-programacao-1-capitulo-2-ano-informatica.pdf> e <http://www.simonsen.br/its/pdf/apostilas/base-tecnica/2/linguagem-de-programacao-2-capitulo-2-ano-informatica.pdf>. Acesso em 12/03/2014.
- LEÃO, MARCELO. Borland Delphi 7 curso completo. Editora Axcel Books, 1ª Edição, 2003.
- VIANA, RAFAEL. Arquivos do Delphi. Web Publicação de 2005. Disponível em <https://sites.google.com/site/estagiariospcsistemas2015/delphi/3---arquivos-do-delphi>. Acesso em 27/01/2016.
- YAMAZACK, WESLEY. Recursos de Compilação no Delphi. Web Publicação. <http://www.devmedia.com.br/recursos-de-compilacao-no-delphi/16964>. Acesso em 25/01/2016